

Laporan TBFO

IF2224 - Teori Bahasa Formal dan Otomata



Disusun Oleh:

Geraldo Artemius - 13524005
Mikhael Andrian Yonatan - 13524051
Junior Natra Situmorang - 13524055
Reynard Nathanael - 13524103
Nicholas Luis Chandra - 13524105

DAFTAR ISI

JUDUL.....	1
DAFTAR ISI.....	2
DAFTAR TABEL.....	3
DAFTAR GAMBAR.....	4
BAB I.....	5
1.1. Parser (Syntax Analyzer).....	5
1.1.1. Jenis Parser.....	5
1.1.2. Ambiguity dalam Parsing.....	7
1.1.3. Peran Parser dalam Compiler.....	7
1.2. Keterkaitan Lexical Analysis and Syntax Analysis.....	7
1.3. Recursive Descent Parsing.....	9
1.3.1. Karakteristik Recursive Descent Parser.....	9
1.3.2. Algoritma Dasar Recursive Descent.....	10
1.3.3. Contoh Grammar Recursive Descent.....	10
1.3.4. Pemetaan Grammar ke Fungsi (Implementasi Konseptual).....	10
1.4. Parse Tree.....	11
1.4.1 Struktur Parse Tree.....	12
1.4.2 Sifat dan Karakteristik Parse Tree.....	12
1.4.3 Fungsi Parse Tree dalam Compiler.....	13
BAB II.....	14
2.1. Struktur Direktori.....	14
2.2. Implementasi Program.....	15
2.3. Aturan Grammar.....	30
BAB III.....	36
3.1 Test Case 1.....	36
3.2 Test Case 2.....	46
3.3 Test Case 3.....	57
3.4 Test Case 4.....	58
3.5 Test Case 5.....	60
3.6 Test Case 6.....	61
3.7 Test Case 7.....	69
BAB IV.....	76
4.1. Kesimpulan.....	76
4.2. Saran.....	76
BAB V.....	77
BAB VI.....	78

DAFTAR TABEL

Tabel 1.2 Perbedaan Lexical Analysis dengan Syntax Analysis.....	9
--	---

DAFTAR GAMBAR

Gambar 1.1 Proses Implementasi Parser.....	5
Gambar 1.1.1 Dua Kategori Parser.....	6
Gambar 1.2 Keterhubungan Antara Lexical Analyzer dan Syntax Analyzer.....	8
Gambar 1.3 Recursive Descent Parsing.....	9
Gambar 1.4 Parse Tree dari String yang Valid.....	12

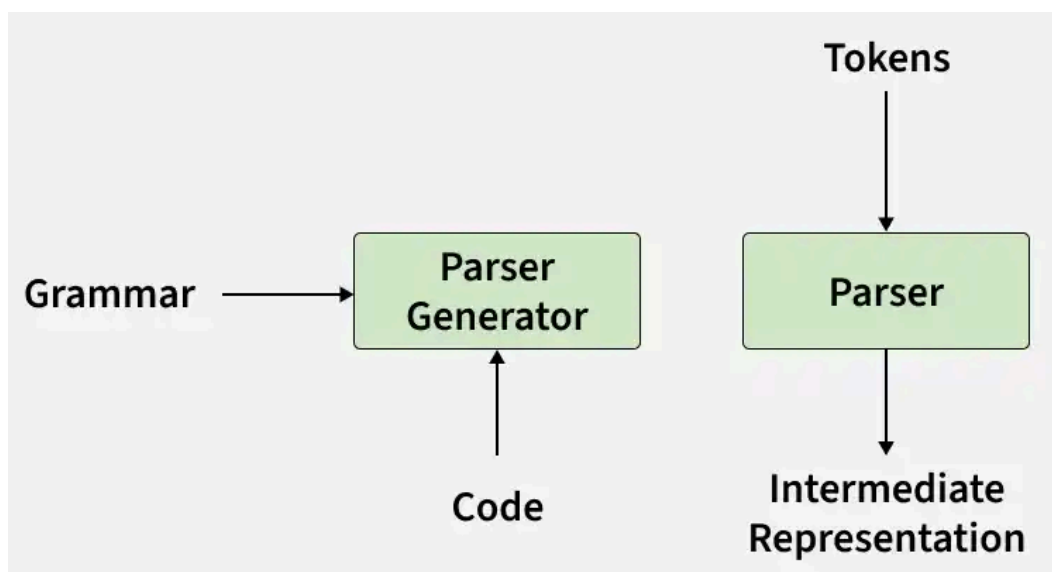
BAB I

TEORI SINGKAT

1.1. Parser (*Syntax Analyzer*)

Parser (*Syntax Analyzer*) merupakan salah satu komponen dalam compiler design yang bertugas untuk menganalisis struktur sintaks dari rangkaian token yang dihasilkan oleh *lexical analyzer*. Parser menerima input berupa token stream dan memeriksa apakah urutan token tersebut sesuai dengan aturan grammar dari bahasa pemrograman yang digunakan. Proses ini dilakukan untuk memastikan bahwa struktur program valid secara sintaksis sebelum masuk ke tahap kompilasi berikutnya.

Dalam prosesnya, parser tidak hanya memeriksa kebenaran susunan token, tetapi juga membentuk representasi struktur hierarkis dari program, seperti **parse tree** atau abstract syntax (AST). Representasi ini membantu compiler dalam memahami hubungan antar elemen dalam program. Jika terdapat ketidaksesuaian dengan grammar, parser akan mendeteksi dan melaporkan terjadinya syntax error sehingga proses kompilasi dapat dihentikan atau diperbaiki.



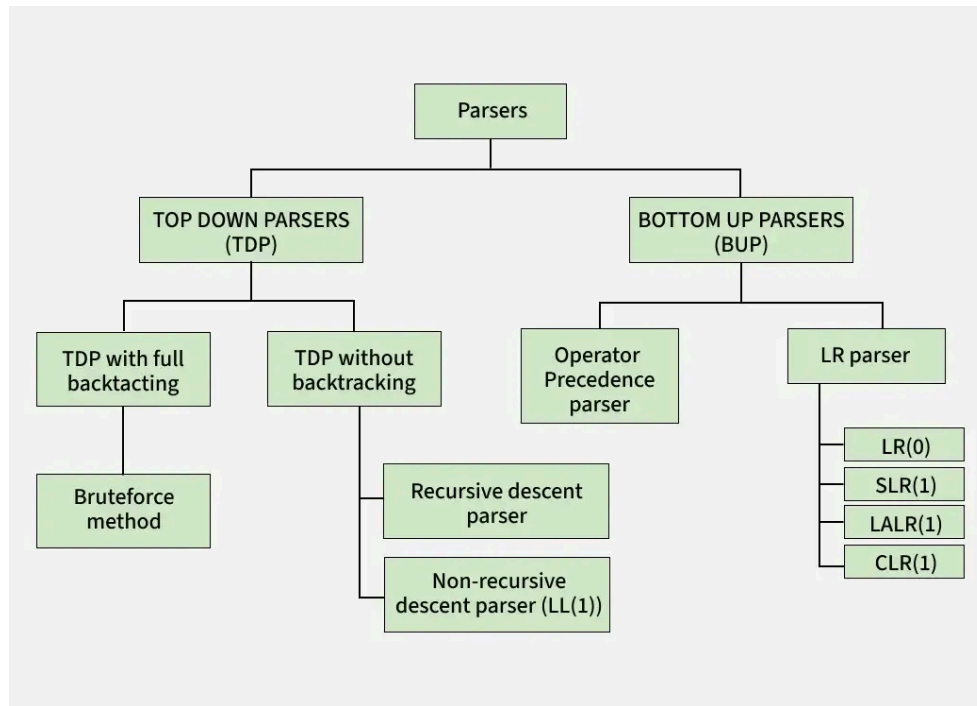
Gambar 1.1 Proses Implementasi Parser

Sumber: <https://www.geeksforgeeks.org/compiler-design/introduction-of-parsing-ambiguity-and-parsers-set-1/>

Secara umum, parser juga menjadi bagian penting dalam proses penerjemahan program karena mengubah representasi linear token menjadi struktur hierarkis yang lebih mudah dianalisis oleh tahap compiler selanjutnya.

1.1.1. Jenis Parser

Parser dalam compiler design secara umum dibagi menjadi dua kategori utama, yaitu top-down parser dan bottom-up parser.



Gambar 1.1.1 Dua Kategori Parser

Sumber: <https://www.geeksforgeeks.org/compiler-design/introduction-of-parsing-ambiguity-and-parsers-set-1/>

• Top Down Parser

Top-down parser membangun **parse tree** dari simbol awal (start symbol) menuju daun (terminal). Proses ini menggunakan pendekatan leftmost derivation, yaitu selalu memperluas non-terminal paling kiri terlebih dahulu. Beberapa jenis top-down parser antara lain:

- Recursive Descent Parser
- Predictive Parser (LL Parser)

Recursive descent parser merupakan metode parsing yang menggunakan fungsi rekursif untuk setiap non-terminal dalam grammar. Sedangkan predictive parser merupakan bentuk yang lebih terstruktur dengan bantuan parsing table untuk menghindari backtracking.

• Bottom Up Parser

Bottom-up parser bekerja dengan cara kebalikan, yaitu membangun **parse tree** dari input token (daun) menuju simbol awal (root). Proses ini menggunakan rightmost derivation in reverse, di mana reduksi dilakukan secara bertahap hingga mencapai start symbol. Jenis bottom-up parser antara lain:

- Shift-Reduce Parser
- LR Parser (SLR, LALR, CLR)

Bottom-up parser umumnya lebih kuat dibandingkan top-down parser karena mampu menangani grammar yang lebih kompleks.

1.1.2. Ambiguity dalam Parsing

Dalam proses parsing, salah satu permasalahan penting yang sering muncul adalah *ambiguity* atau ketaksaan grammar. Grammar dikatakan ambigu apabila terdapat lebih dari satu **parse tree** yang valid untuk satu input string yang sama. Hal ini menyebabkan ketidakjelasan struktur sintaks dari program.

Ambiguity dapat mengakibatkan interpretasi yang berbeda terhadap suatu ekspresi sehingga harus dihindari dalam desain bahasa pemrograman. Beberapa cara yang umum digunakan untuk mengatasi *ambiguity* adalah:

- Menentukan precedence (prioritas operator)
- Menentukan associativity (arah evaluasi operator)
- Memodifikasi grammar agar menjadi unambiguous

1.1.3. Peran Parser dalam Compiler

Parser memiliki peran utama dalam tahap syntax analysis, yaitu memeriksa apakah struktur token sesuai dengan grammar bahasa pemrograman. Parser bertugas menyusun token yang telah dihasilkan oleh lexical analyzer menjadi struktur yang lebih terorganisir seperti **parse tree** atau abstract syntax tree (AST).

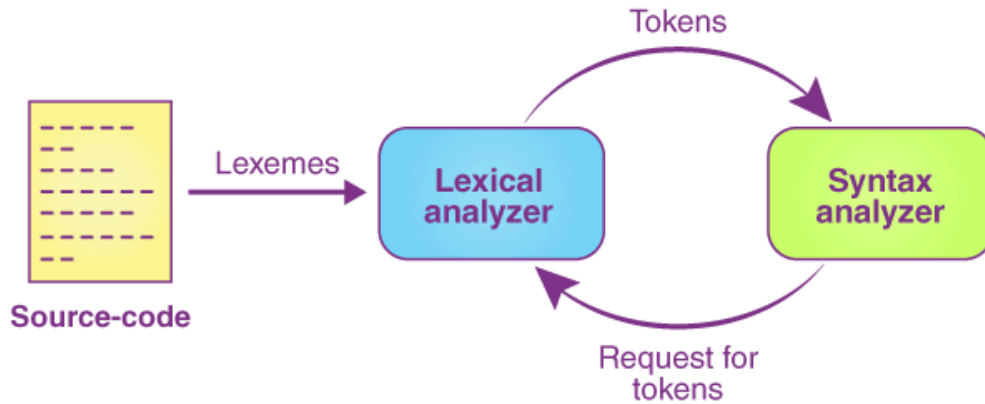
Hasil dari proses parsing ini digunakan untuk merepresentasikan struktur sintaks program secara hierarkis, sehingga dapat dianalisis lebih lanjut pada tahap compiler berikutnya seperti semantic analysis. Secara umum, peran utama parser adalah:

- Memvalidasi struktur sintaks program berdasarkan grammar
- Membentuk representasi hierarkis (parse tree / AST)
- Mendeteksi kesalahan sintaks (syntax error)
- Menyediakan struktur untuk tahap analisis selanjutnya

1.2. Keterkaitan *Lexical Analysis* and *Syntax Analysis*

Lexical analysis dan *syntax analysis* merupakan dua tahap berurutan dalam proses compiler yang saling berkaitan dalam membentuk pemahaman struktur program. Lexical analysis (lexer) berfungsi untuk mengubah source code menjadi token, yaitu unit-unit dasar yang memiliki makna seperti keyword, identifier, operator, dan literal. Token inilah yang kemudian menjadi input bagi tahap berikutnya, yaitu syntax analysis (parser).

Syntax analysis kemudian menggunakan token tersebut untuk memeriksa apakah urutan token sesuai dengan aturan grammar bahasa pemrograman. Pada tahap ini, parser tidak lagi beroperasi pada karakter atau string mentah, melainkan pada hasil abstraksi yang telah dibentuk oleh lexer berupa token stream.



Gambar 1.2 Keterhubungan Antara Lexical Analyzer dan Syntax Analyzer

Source : <https://byjus.com/gate/lexical-analysis/>

Keterkaitan kedua tahap ini dapat dilihat sebagai hubungan input–output yang berurutan, di mana hasil dari lexical analysis menjadi dasar bagi syntax analysis dalam membangun struktur program yang valid. Secara umum, hubungan antara keduanya dapat diringkas sebagai berikut:

- Lexical analysis mengubah source code menjadi token
- Syntax analysis menggunakan token untuk membentuk struktur sintaks
- Token menjadi penghubung utama antara kedua tahap
- Keduanya bekerja berurutan dalam pipeline compiler

Dengan demikian, lexical analysis dan syntax analysis tidak dapat dipisahkan karena keduanya bekerja secara berkesinambungan dalam proses penerjemahan program dari bentuk teks menjadi struktur yang dapat dipahami oleh compiler.

No	Lexical Analysis	Syntax Analysis
1	Proses mengubah karakter pada source code menjadi token	Proses memeriksa token berdasarkan aturan grammar bahasa pemrograman
2	Merupakan fase pertama dalam proses kompilasi	Merupakan fase kedua setelah lexical analysis
3	Dilakukan oleh lexical analyzer (tokenizer)	Dilakukan oleh syntax analyzer (parser)
4	Berfokus pada identifikasi unit dasar seperti keyword, identifier, dan simbol	Berfokus pada struktur dan hubungan antar token
5	Menghasilkan token sebagai output	Menghasilkan parse tree atau AST

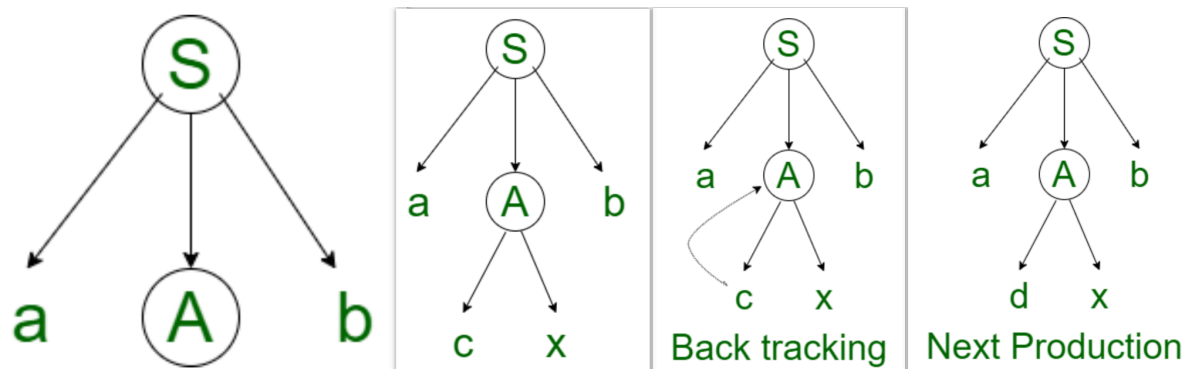
6	Memastikan input terbagi menjadi token yang valid	Memastikan urutan token sesuai grammar
7	Menyediakan input untuk tahap parsing	Memvalidasi struktur sintaks program
8	Digunakan di compiler, interpreter, dan tools analisis teks	Utamanya digunakan dalam compiler dan interpreter

Tabel 1.2 Perbedaan Lexical Analysis dengan Syntax Analysis

1.3. Recursive Descent Parsing

Recursive descent parsing merupakan salah satu metode parsing yang termasuk dalam kategori top-down parser. Metode ini menggunakan pendekatan rekursif untuk membangun **parse tree** dari simbol awal (start symbol) hingga ke token input. Setiap non-terminal dalam grammar direpresentasikan sebagai sebuah fungsi rekursif yang akan dipanggil sesuai dengan aturan produksi yang ada. Dengan demikian, struktur parser secara langsung merefleksikan struktur grammar bahasa yang digunakan.

Pada recursive descent parsing, proses parsing dilakukan dari kiri ke kanan (left-to-right scanning) dan menggunakan konsep leftmost derivation, yaitu selalu mengembangkan non-terminal paling kiri terlebih dahulu. Parser ini bekerja dengan cara mencocokkan token input dengan aturan grammar secara bertahap melalui pemanggilan fungsi-fungsi rekursif.



Gambar 1.3 Recursive Descent Parsing

Sumber: <https://www.geeksforgeeks.org/compiler-design/difference-between-recursive-predictive-descent-parser-and-non-recursive-predictive-descent-parser/>

1.3.1. Karakteristik Recursive Descent Parser

Secara umum, recursive descent parser memiliki beberapa karakteristik utama sebagai berikut:

- Menggunakan pendekatan top-down parsing
- Setiap non-terminal direpresentasikan sebagai fungsi rekursif
- Proses parsing mengikuti struktur grammar secara langsung
- Cocok untuk grammar sederhana seperti LL(1)

- Tidak dapat menangani grammar dengan left recursion tanpa modifikasi grammar

Kelemahan utama metode ini adalah adanya kemungkinan infinite recursion jika grammar mengandung left recursion, sehingga grammar harus diubah terlebih dahulu agar sesuai dengan bentuk yang dapat diproses.

1.3.2. Algoritma Dasar Recursive Descent

Secara konseptual, algoritma recursive descent bekerja dengan cara:

- Menentukan start symbol sebagai fungsi utama parser
- Memanggil fungsi sesuai non-terminal pada grammar
- Melakukan pencocokan token (token matching) dengan input stream
- Jika sesuai, lanjut ke produksi berikutnya
- Jika tidak sesuai, terjadi syntax error atau backtracking (tergantung implementasi)

Setiap fungsi bertanggung jawab untuk memvalidasi satu aturan produksi dalam grammar.

1.3.3. Contoh Grammar Recursive Descent

Misalkan terdapat grammar sederhana untuk ekspresi aritmatika:

```
E  → T E'
E' → + T E' | ε
T  → id
```

Grammar tersebut dapat direpresentasikan ke dalam fungsi-fungsi sebagai berikut:

- $E()$ → memproses ekspresi utama
- $E'()$ → menangani operasi penjumlahan berulang
- $T()$ → menangani terminal (identifikasi)

1.3.4. Pemetaan Grammar ke Fungsi (Implementasi Konseptual)

Grammar recursive descent dapat direpresentasikan secara langsung ke dalam bentuk fungsi rekursif yang sesuai dengan setiap non-terminal. Pendekatan ini membuat proses parsing menjadi lebih intuitif karena struktur program parser mengikuti struktur grammar secara langsung.

- $E()$

Fungsi $E()$ merepresentasikan simbol awal (start symbol) dalam grammar. Fungsi ini pertama-tama akan memanggil $T()$ untuk memproses terminal awal, kemudian dilanjutkan dengan pemanggilan $E'()$ untuk menangani operasi lanjutan sesuai aturan produksi yang ada.

- $E'()$

Fungsi $E'()$ bertugas menangani operasi rekursif pada ekspresi. Jika token saat ini adalah simbol $+$, maka parser akan melakukan pencocokan terhadap token tersebut, kemudian memanggil $T()$ untuk memproses operand berikutnya, dan dilanjutkan dengan pemanggilan

rekursif E') untuk menangani kemungkinan operasi berulang. Jika tidak terdapat simbol +, maka fungsi akan menghasilkan produksi ϵ (epsilon) yang menandakan akhir dari proses ekspresi.

- T()

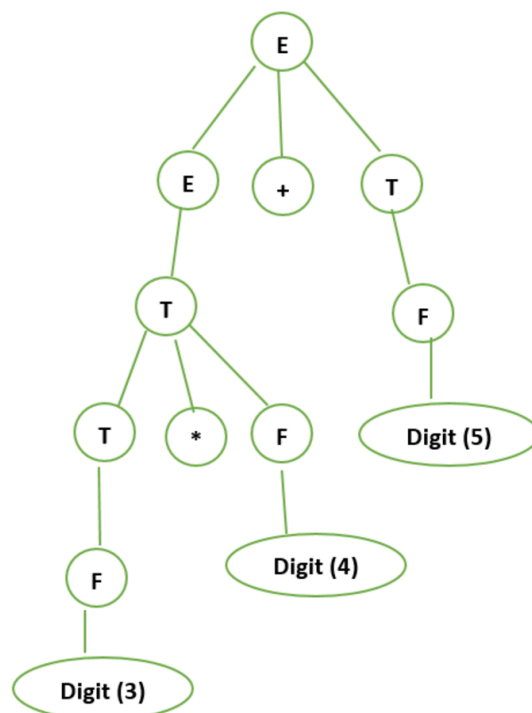
Fungsi T() berperan dalam memproses terminal pada grammar, yaitu token id. Fungsi ini melakukan pencocokan langsung terhadap token input untuk memastikan bahwa token tersebut sesuai dengan aturan grammar.

Secara keseluruhan, pemetaan ini menunjukkan bahwa recursive descent parser bekerja dengan cara menerjemahkan aturan grammar menjadi struktur fungsi rekursif. Pendekatan ini membuat implementasi parser menjadi lebih mudah dipahami, terutama untuk grammar yang sederhana dan tidak kompleks.

1.4. Parse Tree

Parse Tree adalah representasi berbentuk pohon yang menggambarkan bagaimana suatu string dalam bahasa pemrograman diturunkan dari Context-Free Grammar (CFG) melalui serangkaian aturan produksi. Parse tree dibentuk pada tahap syntax analysis oleh parser untuk menunjukkan struktur sintaksis dari input program secara hierarkis.

Parse tree juga dikenal sebagai derivation tree, karena menunjukkan urutan bagaimana simbol awal (start symbol) menghasilkan string input melalui proses derivasi berdasarkan grammar yang telah ditentukan. Dengan demikian, parse tree memberikan gambaran visual mengenai bagaimana aturan grammar digunakan untuk membentuk suatu string yang valid dalam bahasa pemrograman.



Gambar 1.4 Parse Tree dari String yang Valid

Source : <https://www.geeksforgeeks.org/compiler-design/parse-tree-and-syntax-tree/>

1.4.1 Struktur Parse Tree

Secara umum, parse tree memiliki struktur berbentuk pohon berakar (rooted tree) yang menggambarkan proses derivasi suatu string berdasarkan aturan Context-Free Grammar (CFG). Struktur ini digunakan untuk merepresentasikan bagaimana simbol awal (start symbol) diturunkan menjadi terminal melalui serangkaian aturan produksi.

Dalam parse tree, terdapat beberapa komponen utama yang masing-masing memiliki peran tertentu dalam merepresentasikan grammar:

- Root node, yaitu node paling atas yang merepresentasikan simbol awal (start symbol) dari grammar. Node ini menjadi titik awal seluruh proses derivasi dalam parse tree.
- Internal node, yaitu node yang merepresentasikan non-terminal. Node ini masih dapat diturunkan lebih lanjut menjadi simbol lain sesuai aturan produksi yang berlaku.
- Leaf node, yaitu node yang merepresentasikan terminal atau token hasil lexical analysis. Leaf node merupakan hasil akhir dari proses derivasi dan tidak dapat diturunkan lagi.

Selain itu, setiap cabang pada parse tree menunjukkan penerapan aturan produksi grammar yang digunakan dalam proses derivasi. Dengan demikian, struktur parse tree tidak hanya menampilkan hasil akhir string, tetapi juga menggambarkan langkah-langkah pembentukan string secara bertahap dan sistematis.

Struktur ini memastikan bahwa setiap elemen dalam input dapat ditelusuri kembali ke grammar yang membentuknya, sehingga hubungan antara terminal dan non-terminal terlihat jelas dan konsisten.

1.4.2 Sifat dan Karakteristik Parse Tree

Parse tree merupakan representasi berbentuk pohon yang menunjukkan bagaimana suatu string diturunkan dari Context-Free Grammar (CFG) melalui proses derivasi dalam syntax analysis. Struktur ini memperlihatkan hubungan hierarkis antara simbol awal hingga terminal, sehingga setiap langkah pembentukan string dapat ditelusuri secara sistematis. Secara umum, parse tree memiliki beberapa sifat utama sebagai berikut:

- Parse tree bersifat hierarkis, karena setiap node menunjukkan hubungan bertingkat antara simbol grammar dari root hingga leaf. Struktur ini menggambarkan bagaimana start symbol berkembang menjadi string akhir melalui aturan produksi.
- Parse tree bersifat derivatif, yaitu merepresentasikan proses derivasi lengkap dari start symbol menuju string terminal. Setiap cabang pada tree menunjukkan penerapan production rule yang digunakan dalam grammar.
- Setiap node dalam parse tree memiliki peran yang jelas, yaitu root node sebagai start symbol, internal node sebagai non-terminal, dan leaf node sebagai terminal (token) yang dihasilkan dari lexical analysis.

- Parse tree mengikuti aturan grammar secara ketat, di mana setiap ekspansi non-terminal harus sesuai dengan production rules Context-Free Grammar (CFG). Hal ini memastikan bahwa struktur yang dihasilkan valid secara sintaksis.
- Parse tree juga bersifat lengkap secara derivasi, artinya seluruh proses pembentukan string dari awal hingga akhir direpresentasikan tanpa melewatkan langkah produksi apa pun.

1.4.3 Fungsi Parse Tree dalam Compiler

Parse tree merupakan salah satu komponen penting dalam tahap syntax analysis pada compiler, yang digunakan untuk merepresentasikan struktur sintaks suatu program berdasarkan Context-Free Grammar (CFG). Dalam proses kompilasi, parse tree dibentuk setelah token dihasilkan oleh lexical analyzer dan diverifikasi oleh parser untuk memastikan kesesuaian dengan aturan grammar. Secara umum, parse tree memiliki beberapa fungsi utama dalam compiler, yaitu:

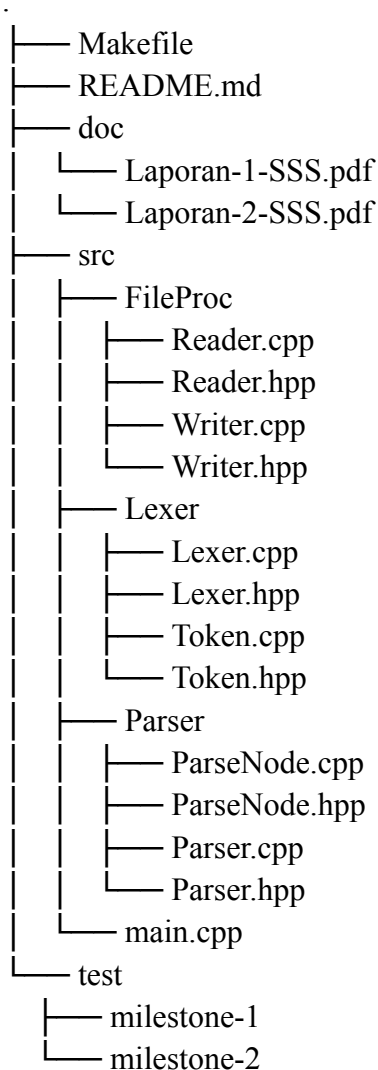
- Parse tree digunakan untuk merepresentasikan struktur sintaks program secara hierarkis, sehingga hubungan antara terminal dan non-terminal dalam grammar dapat terlihat dengan jelas. Representasi ini membantu compiler memahami bagaimana suatu program dibentuk dari aturan grammar yang berlaku.
- Parse tree berfungsi untuk memvalidasi kebenaran sintaks program, yaitu memastikan bahwa urutan token yang dihasilkan oleh lexical analysis sesuai dengan grammar bahasa pemrograman. Jika tidak sesuai, maka parser akan mendeteksi adanya syntax error.
- Parse tree menjadi dasar untuk tahap analisis lanjutan seperti semantic analysis dan code generation, karena struktur hierarkisnya memudahkan compiler dalam memahami hubungan antar elemen program.
- Parse tree juga digunakan sebagai representasi antara (intermediate representation) yang menjembatani source code dengan proses transformasi ke bentuk lain seperti Abstract Syntax Tree (AST) atau intermediate code.

Selain itu, parse tree membantu compiler dalam melakukan penelusuran struktur program secara sistematis, karena setiap node dalam tree merepresentasikan simbol grammar yang digunakan dalam proses derivasi. Dengan demikian, parse tree tidak hanya berfungsi sebagai alat visualisasi, tetapi juga sebagai struktur penting dalam proses analisis sintaks.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Struktur Direktori



Sekarang struktur folder kami, masih sama dengan laporan sebelumnya, secara garis besar :

- Folder doc berisi dokumen laporan Tugas Besar dalam bentuk dokumen PDF.
- Folder src berisi *source code* program dalam bahasa C++ yang mencakup implementasi mesin DFA (Lexer.cpp, Lexer.hpp) pada folder Lexer, definisi token (Token.cpp, Token.hpp) pada folder Lexer juga, pemrosesan file (Reader.cpp, Reader.hpp, Writer.cpp, Writer.hpp) pada folder FileProc, integrasi seluruh modul (main.cpp), dan sekarang ada folder Parser yang mencakup implementasi modul *parse tree* (ParseNode.hpp, ParseNode.cpp, Parser.cpp, dan Parser.hpp).

- Folder test berisi file pengujian, baik berupa data masukan (source code bahasa Arion) maupun data keluaran hasil pemrosesan token. Terdapat folder milestone-1 dan milestone-2 di dalamnya untuk menentukan *test case* per *milestone*.
- File Makefile berisi kumpulan instruksi kompilasi untuk mempermudah proses build source code C++ menjadi program executable secara otomatis.

2.2. Implementasi Program

1. ParseNode.hpp & ParseNode.cpp

ParseNode.hpp

```
class ParseNode {
private:
    string name; // Label Non-Terminal (contoh: "<program>")
    Token token; // Token Terminal
    bool isTerminal;
    vector<ParseNode*> children;
public:
    // Constructor Non-Terminal
    ParseNode(const string& name);
    // Constructor Terminal
    ParseNode(const Token& token);
    ~ParseNode();
    void addChild(ParseNode* child);
    string getName() const;
    Token getToken() const;
    bool getIsTerminal() const;
    const vector<ParseNode*>& getChildren() const;
    void printTree(const string& prefix = "", bool isLast = true, bool
isRoot = true) const;
};
```

Pada ParseNode.hpp, implementasi berfungsi sebagai pondasi struktur data utama untuk membangun dan merepresentasikan *parse tree*. Bagian ini mendefinisikan atribut token untuk merekam data objek terminal, parameter name bertipe string untuk melabeli simpul non-terminal (seperti "<program>"), serta penanda boolean isTerminal untuk memudahkan verifikasi tipe simpul. Selain itu, didefinisikan tempat penampungan percabangan ke derajat yang lebih rendah dalam bentuk struktur vektor pointer bernama children.

ParseNode.cpp

```
ParseNode::ParseNode(const string& name){
```

```

    INPUT : Referensi nama bertipe string yang tidak boleh dimodifikasi
    OUTPUT : Objek ParseNode berupa non-terminal
    CARA KERJA :
    1. Mengisi variabel internal name dengan string yang diterima.
    2. Mengatur flag isTerminal menjadi false.
    3. Menginisialisasi tipe token ke status UNKNOWN.
}

ParseNode::ParseNode(const Token& token){
    INPUT : Referensi ke Token yang tidak boleh dimodifikasi
    OUTPUT : Objek ParseNode berupa Terminal
    CARA KERJA :
    1. Mengosongkan variabel name.
    2. Menyimpan objek token ke variabel internal.
    3. Mengatur flag isTerminal menjadi true.
}

ParseNode::~~ParseNode(){
    INPUT : Objek yang ingin di-destruct
    OUTPUT : Memori dibersihkan
    CARA KERJA :
    1. Melakukan iterasi melalui seluruh pointer di dalam vektor
        children.
    2. Memanggil operator delete pada setiap anak secara rekursif.
}

void ParseNode::addChild(ParseNode* child){
    INPUT : Pointer ke node yang ingin dijadikan child suatu node.
    OUTPUT : Penambahan elemen ke dalam list children
    CARA KERJA :
    1. Melakukan validasi apakah pointer child tidak bernilai nullptr.
    2. Memasukkan pointer tersebut ke dalam vektor children.
}

String ParseNode::getName() const {
    INPUT : -
    OUTPUT : String nama dari node
    CARA KERJA :
    1. Mengembalikan nilai dari variabel internal name.
}

Token ParseNode::getToken() const {
    INPUT : -
    OUTPUT : Objek Token yang disimpan di Node
    CARA KERJA :
    1. Mengembalikan data token milik node tersebut.
}

```

```

bool ParseNode::getIsTerminal() const {
    INPUT : -
    OUTPUT : Boolean status terminal node
    CARA KERJA :
    1. Mengembalikan nilai true jika node adalah terminal (daun), atau
       false jika non-terminal.
}

const vector<ParseNode*>& ParseNode::getChildren() const {
    INPUT : -
    OUTPUT : Referensi ke vektor berisi pointer anak-anak node
    CARA KERJA :
    1. Mengembalikan kumpulan node anak yang terhubung dengan node saat
       ini.
}

void ParseNode::printTree(const string& prefix, bool isLast, bool
isRoot) const {
    INPUT : String prefix untuk indentasi, status anak terakhir, dan
           status root
    OUTPUT : Tampilan struktur pohon di CLI
    CARA KERJA :
    1. Mencetak simbol struktur (├─ atau └─) dan prefix jika node
       bukan root.
    2. Jika node adalah non-terminal, cetak namanya. Jika terminal,
       cetak tipe token dan nilainya.
    3. Melakukan rekursi ke setiap anak dalam vektor children dengan
       memperbarui prefix dan status isLast untuk membentuk visualisasi
       pohon yang rapi
}

```

Pada `ParseNode.cpp`, implementasi berisi konstruktor untuk menginisialisasi simpul baik sebagai non-terminal (menggunakan nama) maupun terminal (menggunakan objek `Token`), beserta destruktur yang bertugas membebaskan memori secara rekursif dari seluruh simpul anak di `children`. Selain itu, terdapat fungsi `addChild` untuk mengaitkan koneksi antar simpul dalam struktur pohon. Untuk mendukung analisis data, kelas ini menyediakan berbagai metode getter untuk mengakses informasi nama, token, dan struktur hierarki anak. Terakhir, terdapat metode `printTree` yang mengimplementasikan algoritma penelusuran pohon untuk memvisualisasikan struktur parse tree.

2. Parser.hpp & Parser.cpp

Parser.hpp

```

class Parser {
private:
    vector<Token> tokens;
    int pos;

    Token currentToken() const;
    Token peek(int offset) const;
    bool isAtEnd() const;
    void advance();
    Token match(TokenType expected);

    ParseNode* parseProgram();
    ParseNode* parseProgramHeader();
    ParseNode* parseBlock();

    ParseNode* parseDeclarationPart();
    ParseNode* parseConstDeclaration();
    ParseNode* parseConstant();
    ParseNode* parseTypeDeclaration();
    ParseNode* parseVarDeclaration();
    ParseNode* parseIdentifierList();
    ParseNode* parseSubprogramDeclaration();
    ParseNode* parseProcedureDeclaration();
    ParseNode* parseFunctionDeclaration();
    ParseNode* parseFormalParameterList();
    ParseNode* parseParameterGroup();

    ParseNode* parseType();
    ParseNode* parseArrayType();
    ParseNode* parseRange();
    ParseNode* parseEnumerated();
    ParseNode* parseRecordType();
    ParseNode* parseFieldList();
    ParseNode* parseFieldPart();

    ParseNode* parseCompoundStatement();
    ParseNode* parseStatementList();
    ParseNode* parseStatement();
    ParseNode* parseVariable();
    ParseNode* parseComponentVariable();
    ParseNode* parseIndexList();
    ParseNode* parseAssignmentStatement();
    ParseNode* parseIfStatement();
    ParseNode* parseCaseStatement();
    ParseNode* parseCaseBlock();
    ParseNode* parseWhileStatement();
    ParseNode* parseRepeatStatement();
    ParseNode* parseForStatement();

```

```

ParseNode* parseProcedureFunctionCall();
ParseNode* parseParameterList();

ParseNode* parseExpression();
ParseNode* parseSimpleExpression();
ParseNode* parseTerm();
ParseNode* parseFactor();
ParseNode* parseRelationalOperator();
ParseNode* parseAdditiveOperator();
ParseNode* parseMultiplicativeOperator();

public:
    Parser(const vector<Token>& tokens);
    ParseNode* parse();
};

```

File Parser.hpp mendefinisikan kelas Parser yang berfungsi sebagai penganalisis sintaksis menggunakan metode Recursive Descent Parsing, di mana setiap metode privat di dalamnya merepresentasikan aturan produksi sesuai spesifikasi, mulai dari struktur program utama, blok deklarasi, hingga logika pernyataan kondisional dan ekspresi aritmatika. Header ini menyediakan beberapa utilitas untuk membaca urutan token melalui fungsi utilitas seperti match dan advance, yang nantinya diimplementasikan di dalam file Parser.cpp untuk membangun struktur *parse tree* dengan menghasilkan objek ParseNode sebagai representasi dari setiap unit sintaksis yang valid.

Parser.cpp

```

Parser::Parser(const vector<Token>& tokens) {
    INPUT : Vektor berisi token-token hasil dari lexical analysis
    OUTPUT: Objek parser dengan array token dan posisi awal di indeks 0
    CARA KERJA:
    1. Simpan vektor token masukan ke variabel internal
    2. Inisialisasi indeks posisi pembacaan token ke 0
}

Token Parser::currentToken() const {
    INPUT : Posisi parser saat ini
    OUTPUT: Objek Token pada indeks saat ini, atau token terakhir jika
            sudah di ujung list.
    CARA KERJA:
    1. Cek apakah posisi sudah mencapai/melewati jumlah token (isAtEnd)
    2. Jika sudah di akhir, kembalikan token paling belakang
    3. Jika belum, kembalikan token yang ditunjuk oleh indeks posisi
        sekarang
}

```

```

Token Parser::peek(int offset) const {
    INPUT : Offset (jumlah indeks ke depan dari posisi indeks saat ini)
    OUTPUT: Objek Token pada (posisi + offset), atau token terakhir
           jika offset melewati batas list
    CARA KERJA:
    1. Hitung indeks target (pos + offset)
    2. Pastikan target tidak melampaui jumlah total token
    3. Jika melewati batas, kembalikan token terakhir ujung array.
}

bool Parser::isAtEnd() const {
    INPUT : Posisi parser saat ini dan panjang list token
    OUTPUT: true jika seluruh list token telah terbaca.
    CARA KERJA:
    1. Bandingkan nilai indeks posisi saat ini dengan ukuran array dari
       token
    2. Kembalikan true jika pos sudah  $\geq$  ukuran array
}

void Parser::advance() {
    INPUT : Posisi parser saat ini
    OUTPUT: State posisi parser maju 1 langkah
    CARA KERJA:
    1. Cek terlebih dahulu menggunakan isAtEnd()
    2. Jika list token belum habis, increment (+) indeks posisi
}

Token Parser::match(TokenType expected) {
    INPUT : Tipe token ekspektasi yang diharapkan menurut grammar
    OUTPUT: Objek token jika berhasil dicocokkan, atau membentuk token
           UNKNOWN jika gagal
    CARA KERJA:
    1. Periksa apakah pos tidak di akhir baris token dan tipe token
       saat ini sama dengan parameter "expected"
    2. Jika BENAR, simpan token saat ini bertipe ke var sementara,
       lakukan advance(), dan kembalikan token.
    3. Jika SALAH (Panic Mode), cetak di layar terminal pesan Syntax
       Error
    4. Bentuk token palsu bernilai UNKNOWN
}

ParseNode* Parser::parseProgram() {
    INPUT : Token stream parser pada rule awal
    OUTPUT: Abstract Syntax Tree (AST) node dari "<program>"
    CARA KERJA:
    1. Buat ParseNode dengan nama "<program>"
    2. Tambahkan node anak secara sekuensial dengan memanggil
       parseProgramHeader()
}

```

```

3. Tambahkan node anak secara sekuensial dengan memanggil
   parseDeclarationPart()
4. Tambahkan node anak secara sekuensial dengan memanggil
   parseCompoundStatement()
5. Cocokkan PERIOD dan buat simpul
}

ParseNode* Parser::parseProgramHeader() {
    INPUT : Posisi stream untuk awalan deklarasi sebuah program
    OUTPUT: Node cabang "<program-header>"
    CARA KERJA:
    1. Buat ParseNode baru "<program-header>"
    2. Lakukan match untuk keyword PROGRAM, identifier, kemudian
       SEMICOLON
    3. Buat token yang berhasil dicocokkan sebagai child node
}

ParseNode* Parser::parseBlock() {
    INPUT : Posisi stream untuk block dari fungsi/prosedur
    OUTPUT: Node cabang "<block>"
    CARA KERJA:
    1. Bangun ParseNode "<block>"
    2. Lakukan sub-pemanggilan rekursif ke parseDeclarationPart() dan
       parseCompoundStatement()
    3. Tambahkan return pohon AST sebagai turunan node
}

ParseNode* Parser::parseDeclarationPart() {
    INPUT : Posisi awalan list deklarasi (variabel, konstan, tipe, dsb)
    OUTPUT: Node root untuk "<declaration-part>"
    CARA KERJA:
    1. Cek pakai perulangan while tipe token untuk tiap bagian
    2. Jika ada CONST, parseConstDeclaration()
    3. Jika ada TYPE, parseTypeDeclaration()
    4. Jika ada VAR, parseVarDeclaration()
    5. Jika ada FUNCTION / PROCEDURE, parseSubprogramDeclaration()
    6. Ubah semuanya menjadi berurutan sebagai children.
}

ParseNode* Parser::parseConstDeclaration() {
    INPUT : Token awalan deklarasi konstanta
    OUTPUT: Node tipe "<const-declaration>"
    CARA KERJA:
    1. Buat node, match token keyword CONST
    2. Loop do-while untuk menangkap identifier → operator equal →
       parsing constant value → ;
    3. Berhenti saat identifier const berikutnya sudah tidak ditemukan
}

```

```

ParseNode* Parser::parseConstant() {
    INPUT : Objek parser menghadap ke nilai konstan
    OUTPUT: Node turunan untuk mendeskripsikan ekspresi konstan
            ("<constant>")
    CARA KERJA:
    1. Cek tipe token saat ini (apakah dia String, Char). Jika ya,
       match dan rakit.
    2. Jika ada modifikator Plus/Minus, tangkap modifikator terlebih
       dahulu.
    3. Baru cek tipe primitif selanjutnya (Identifier, Int, Real).
       Match token.
    4. Jika tak dikenali satupun, match jadi token UNKNOWN dari Panic
       Mode.
}

ParseNode* Parser::parseTypeDeclaration() {
    INPUT : Token aliran saat awalan deklarasi tipe (TYPE)
    OUTPUT: Node cabang untuk "<type-declaration>"
    CARA KERJA:
    1. Match keyword TYPE
    2. Looping minimal 1 kali menangkap Identifier = parseType(),
       implementasi menggunakan do while;
}

ParseNode* Parser::parseVarDeclaration() {
    INPUT : Token aliran deklarasi variabel (VAR)
    OUTPUT: Node cabang "<var-declaration>"
    CARA KERJA:
    1. Match keyword VAR
    2. Looping minimal 1 kali: parseIdentifierList(), match Token
       dengan colon, parseType(), dan terakhir match dengan Token
       semicolon
}

ParseNode* Parser::parseIdentifierList() {
    INPUT : List identifer yang dipisah dengan koma (ID, ID, ...)
    OUTPUT: Pohon sintaks yang memuat rangkaian identifier
    CARA KERJA:
    1. Ambil token ID pertama
    2. Loop selama ada COMMA, ambil token koma lalu identifier-nya.
}

ParseNode* Parser::parseSubprogramDeclaration() {
    INPUT : Pencabangan di deklarasi yang bisa jadi PROSEDUR / FUNGSI
    OUTPUT: Node root untuk "<subprogram-declaration>"
    CARA KERJA:
    1. Cek jika tipenya PROCEDURE maka turun ke

```

```

    parseProcedureDeclaration()
2. Atau jika FUNCTION turun ke parseFunctionDeclaration()
3. Di luar itu bangkitkan token ERROR/UNKNOWN.
}

ParseNode* Parser::parseProcedureDeclaration() {
    INPUT : Rule program untuk membaca kerangka PROCEDURE
    OUTPUT: Node AST bagi "<procedure-declaration>"
    CARA KERJA:
    1. Cocokkan word PROCEDURE dan ID.
    2. Jika terdapat kurung '(' maka muat parseFormalParameterList().
    3. Simpan node Titik Koma (;). Lanjut muat parseBlock() dan ditutup
        Titik Koma.
}

ParseNode* Parser::parseFunctionDeclaration() {
    INPUT : Aliran kerangka FUNCTION
    OUTPUT: Node subpohon abstrak untuk struktur fungsi tsb
    CARA KERJA:
    1. Tangkap FUNCTION dan ID. Jika ada parameter list (LPARENT),
        diparse.
    2. Match Tanda Titik Dua (:) lalu ID untuk tipe data hasil fungsi.
        Titik koma.
    3. Compile isinya via parseBlock() lalu ditutup Titik koma.
}

ParseNode* Parser::parseFormalParameterList() {
    INPUT : Bagian stream dalam tanda kurung parameter ()
    OUTPUT: Node subtree parameter
    CARA KERJA:
    1. Ambil LPARENT. Lalu load sub-bagian dengan parseParameterGroup()
    2. Loop tiap parameter group jika dibatasi titik-koma
    3. Tutup kurung RPARENT
}

ParseNode* Parser::parseParameterGroup() {
    INPUT : Satu pasang definisi dari argumen seperti: (a,b: integer)
    OUTPUT: Anak dari node formal parameter list.
    CARA KERJA:
    1. Rangkai identifier list dan titik dua (:)
    2. Deteksi tipe untuk parseArrayType() jika ARRAY, lainnya jadikan
        ID konvensional.
}

ParseNode* Parser::parseType() {
    INPUT : Token aliran saat definisi/deklarasi tipe dilakukan
    OUTPUT: Pohon sintaks untuk identitas/jenis turunan tipe ("<type>")
    CARA KERJA:

```

1. Lakukan if-statement pencabangan untuk menentukan mana dari: `parseArrayType()` (jika ARRAY), `parseEnumerated()` (jika kurung buka), `parseRecordType()` (jika RECORD).
2. Jika saat ini Identifier dan berikutnya Titik, maka terjun ke `parseRange()`.
3. Jika Literal primitif (angka real, intcon) juga lempar sebagai Range primitive.
4. Lainnya lempar exception UNKNOWN.

}

ParseNode* Parser::parseArrayType() {

INPUT : Stream token di saat mendefinisikan tipe data Array

OUTPUT: AST untuk tipe struktur Array

CARA KERJA:

1. Cocokkan kata ARRAY dan buka kurung siku [
2. Identifikasi index nya lalu parsing `parseRange()` untuk dimensinya
3. Tutup siku, kata deklarsi OF, dan parsing base type nya via `parseType()`

}

ParseNode* Parser::parseRange() {

INPUT : List range seperti 1..10

OUTPUT: Node ekspresi limit bounds

CARA KERJA:

1. Muat konstanta pertama
2. Muat keyword periode dua kali
3. Muat konstanta yang menjadi stop limit range

}

ParseNode* Parser::parseEnumerated() {

INPUT : Struct custom enumerasi di pascal

OUTPUT: Set himpunan dari enumerasi

CARA KERJA:

1. Match LPARENT, match deret element dengan pemisah COMMA layaknya param list.
2. Tutup RPARENT

}

ParseNode* Parser::parseRecordType() {

INPUT : Sub stream sebuah *record*

OUTPUT: AST komposit record type beserta list child nya.

CARA KERJA:

1. Tangkap RECORD. Kemudian generate isi dengan delegasi `parseFieldList()`
2. Akhiri node END.

}

```

ParseNode* Parser::parseFieldList() {
    INPUT : Baris fields dari sebuah entity tipe record
    OUTPUT: Node pohon sintaks
    CARA KERJA:
    1. Loop menyambung rangkaian parseFieldPart() sambil merantai
        SEMICOLON (;) yang mengikutinya.
}

ParseNode* Parser::parseFieldPart() {
    INPUT : Elemen deklarasi layaknya param atau var dalam record
    OUTPUT: Node Field individual
    CARA KERJA:
    1. Dapatkan list identifier, titik dua (;), selanjutnya ambil
        type-nya dengan parseType().
}

ParseNode* Parser::parseCompoundStatement() {
    INPUT : Token untuk blok BEGIN - END
    OUTPUT: Blok eksekusi compound statement
    CARA KERJA:
    1. Start blok BEGIN. Kemudian tampung command/instruksi lewat
        parseStatementList().
    2. Sudahi lewat blok END.
}

ParseNode* Parser::parseStatementList() {
    INPUT : List pernyataan sekuensial yang dibatasi Titik Koma (;)
    OUTPUT: Node kompilasi dari banyak statement tersebut.
    CARA KERJA:
    1. Baca statement pertama dengan "parseStatement()".
    2. Lanjutkan dengan me-loop dan membaca titik-koma tiap diikuti
        statement lain.
}

ParseNode* Parser::parseStatement() {
    INPUT : Subprogram bersiap membaca statemnt.
    OUTPUT: Node tipe "<statement>" dan seluruh keturunannya
    CARA KERJA:
    1. Bentuk ParseNode jenis label list baris code. Ambil tipe token
        saat ini
    2. Temukan alur menggunakan percabangan if-else sesuai kontrol
        strukturnya (If, Case, While, Repeat, For)
    3. Jika identifier biasa, gunakan peek() sejauh +1
    4. Tergantung apa operator pasca-identifier tersebut (seperti
        Assign atau kurung panggil Parameter fungsi), serahkan eksekusi
        ke parseAssignmentStatement() atau parseProcedureFunctionCall().
}

```

```

ParseNode* Parser::parseVariable() {
    INPUT : Identifier dari nama variabel maupun modifikasinya
    OUTPUT: Node dari Variable lengkap
    CARA KERJA:
    1. Tangkap sebuah identifer di root node ini.
    2. Selama ia diikuti LBRAK `[' atau PERIOD `.`, rekursifkan
        component di bawah parseComponentVariable().
}

ParseNode* Parser::parseComponentVariable() {
    INPUT : Akses suffix variabel list atau entity struct (. atau [])
    OUTPUT: Child Component Var
    CARA KERJA:
    1. Jika itu LBRAK (Array idx), buat branch parseIndexList().
    2. Jika itu titik (.) (Record attr), grab indentifier langsung.
}

ParseNode* Parser::parseIndexList() {
    INPUT : Rentetan ID/Literals akses matriks yg dipisah Koma
    OUTPUT: AST indeks target matriks/array
    CARA KERJA:
    1. Cek ID atau konstan int lalu match. Jika ada COMMA lanjut loop
        rekursif parameter dirinya.
}

ParseNode* Parser::parseAssignmentStatement() {
    INPUT : Ekspresi set sebuah var ke sesuatu
    OUTPUT: Node Assignment
    CARA KERJA:
    1. Memanggil fungsi parseVariable() karena sebelum := atau variabel
        yang harus dicek.
    2. match operator BECOMES (:=).
    3. Lalu memanggil sisi kanannya lewat fungsi parseExpression().
}

ParseNode* Parser::parseIfStatement() {
    INPUT : Sebuah blok conditional IF THEN ELSE
    OUTPUT: Pohon eksekusi IF
    CARA KERJA:
    1. Muat IF, masuk eval-boolean dari parseExpression(), muat kata
        kunci THEN.
    2. Parsing blok program jika trueth dengan parseStatement().
    3. Jika ternyata ada ELSE, buat node child untuk else statment-nya.
}

ParseNode* Parser::parseCaseStatement() {
    INPUT : Pemilihan CASE OF di pascal
    OUTPUT: Pohon eksekusi switch-case (case-statement)
}

```

CARA KERJA:

1. Ambil CASE beserta kondisi value di `parseExpression()`-nya + tutup word `OF`.
2. Eksekusi isinya pakai `parseCaseBlock()`. Tutup scope blok dengan `END`.

}

`ParseNode* Parser::parseCaseBlock() {`

INPUT : Tiap kondisi ketika value `X`

OUTPUT: Tree yang memetakan value terhadap aksinya

CARA KERJA:

1. Parse label constantnya misal " 1,2: <aksi> "
2. Ambil COLON (:), eksekusi the statement lewat `parseStatement()`.
3. Ulang / rekursikan jika dipisah pakai pemisah titik koma ; tapi belum masuk ke blok final.

}

`ParseNode* Parser::parseWhileStatement() {`

INPUT : Blok looping While

OUTPUT: AST yang me-represent While loop

CARA KERJA:

1. Match WHILE identifier. Lalu periksa ekspresi while loop dengan `parseExpression()`.
2. Match DO. Kemudian bangun blok statment di bawahnya `parseStatement()`.

}

`ParseNode* Parser::parseRepeatStatement() {`

INPUT : Looping do-while atau Repat-Until

OUTPUT: AST perulangan dari Repeat

CARA KERJA:

1. Match REPEAT.
2. Lalu selanjutnya panggil method `parseStatementList()`.
3. Match termination token UNTIL dilanjut parsing boolean di ekspresinya `parseExpression()`.

}

`ParseNode* Parser::parseForStatement() {`

INPUT : Loop counting FOR loop Bahasa Arion

OUTPUT: AST pohon instruksi for loop

CARA KERJA:

1. Lakukan match assignment awal iterator `ID := Value var` pakai `ParseExpression`.
2. Tangkap jenis TO atau DOWNTO loop
3. Set expression ujungnya untuk limit iterasi, tangkap DO word dan eksekusi child block parsingnya.

}

```

ParseNode* Parser::parseProcedureFunctionCall() {
    INPUT : Calling syntax untuk fungsi maupun prosedur call
    OUTPUT: AST Tree panggilan dengan argumen args
    CARA KERJA:
    1. Match ID dari method target.
    2. Cek eksistensi argumen yang dilingkupi kurung (). Kalau isinya
        bukan RPARENT langsung panggil parseParameterList() untuk
        argumennya sebelum ditutup.
}

ParseNode* Parser::parseParameterList() {
    INPUT : Args yang di passing
    OUTPUT: Deretan arg (ekspresi-ekspresi yang diberikan)
    CARA KERJA:
    1. Ambil parseExpression().
    2. Loop bila dipisahkan dengan koma tangkap lebih banyak
        parseExpression().
}

ParseNode* Parser::parseExpression() {
    INPUT : Operasi statement matematika/logika
    OUTPUT: Resolusi struktur matematika dari ekspresi
    CARA KERJA:
    1. Memanggil fungsi parseSimpleExpression().
    2. Cek token relasional (=, <, >, dsb) jika ada, terus operasikan
        bersama operand setelahnya dengan call berantai.
}

ParseNode* Parser::parseSimpleExpression() {
    INPUT : String matematika lanjutan level term
    OUTPUT: Node math branch
    CARA KERJA:
    1. Cek Prefix modifier pos/min jika exists.
    2. Buka term menggunakan parseTerm()
    3. Jika diikuti operan OR/tambah/kurang, terus tambahkan
        parseTerm() iteratif secara sejajar.
}

ParseNode* Parser::parseTerm() {
    INPUT : Ekspresi term math operator seperti bagi,kali, dan
        sebagainya
    OUTPUT: Node list terminal dari sebuah Term
    CARA KERJA:
    1. Ambil factor primitive lewat parseFactor().
    2. Ulang parseFactor() selama current token bertipe TIMES, RDIV,
        IDIV, dan sebagainya lalu memanggil fungsi
        parseMultiplicativeOperator() dilanjutkan dengan parseFactor()
}

```

```

ParseNode* Parser::parseFactor() {
    INPUT : Value paling bawah
    OUTPUT: Node Factor Math
    CARA KERJA:
    1. Cek apakah dia const primitif (Intcon, String), dan assign token
       value di Node daun primitif ini.
    2. Jika ada kurawal (), memanggil fungsi parseExpression()
       dilanjutkan dengan tutup kurung.
    3. Jika tipe ada di modifier bool seperti NOT jabarkan rekursif
       isinya
    4. Identifier di parse sebagai param fungsi call ID() jika terdapat
       next=LPARENT.Dan parseVariable() normal bila tidak ada next nya
       .
    5. UNKNOWN jika tidak lolos kondisi.
}

ParseNode* Parser::parseRelationalOperator() {
    INPUT : Simbol perbandingan Relational (<,<=, dsb)
    OUTPUT: Operator sebagai daun / terminal root di math tree
    CARA KERJA:
    1. Cek relasional EQL|GBR|LSS. Kemudian match saja jika ada.
}

ParseNode* Parser::parseAdditiveOperator() {
    INPUT : Simbol operator tambah logic/math (OR, +, -)
    OUTPUT: Node Child simbol tersebut
    CARA KERJA:
    1. Cek Additivity (+ , -, OR) kemudian load Node pakai Match
}

ParseNode* Parser::parseMultiplicativeOperator() {
    INPUT : Simbol operasi komputasi tier-2 (*,/ MOD dll)
    OUTPUT: Node Child simbol
    CARA KERJA:
    1. Cek tipe jika token nya = `*`, `MOD`, `DIV`, dsb . Kemudian
       panggil Match().
}

```

File Parser.cpp mengimplementasikan logika utama dari kelas Parser untuk menganalisis sintaksis dan membangun *parse tree* menggunakan Recursive Descent Parser. Melalui pemanggilan fungsi rekursif untuk setiap aturan produksi (seperti parseProgram() hingga parseFactor()), program akan mengevaluasi sintaks dan membangun sebuah Parse Tree yang merepresentasikan hierarki program. Selain itu, file ini mengandalkan fungsi match() sebagai validator terminal sekaligus mekanisme error

handling, jika token tidak sesuai dengan tata bahasa yang diharapkan, program akan masuk ke Panic Mode dan menampilkan pesan peringatan Syntax Error.

2.3. Aturan Grammar

No	Nama Node	Aturan Produksi	Contoh
1	<program>	<program-header> + <declaration-part> + <compound-state> + period	program myProgram; const n = 10; var i: integer; begin i := 0; end.
2	<program-header>	programsy + identifier + semicolon	program Hello;
3	<declaration-part >	(<const-declaration>)* + (<type-declaration>)* + (<var-declaration>)* + (<subprogram-declaration>)*	const maxScore = 100; pi = 3.14; type Point = record x: integer; y: integer; end; var a, b: integer; p: Point;
4	<const-declaration>	constsy + (ident + eql + <constant> + semicolon) ⁺	const INT == 100; REAL == 10.0; CHAR == 'a'; STRING == 'abcd';

5	<constant>	charcon string [(plus minus)? + (ident intcon realcon)]	'a' 'abcd' 10 +10.0 -a
6	<type-declaration>	typesy + (ident + eql + type + semicolon) ⁺	type Range == 1..10; type Month == (Jan, Feb, Mar, Apr);
7	<var-declaration>	varsy + (<identifier-list> + colon + <<type> + semicolon) ⁺	var a, b: integer;
8	<identifier-list>	ident (comma + ident) ⁺	a, b, c
9	<type>	ident <array-type> <range> <enumerated> <record-type>	integer, array[1..10] of integer
10	<array-type>	arraysy + lbrack + (<range> ident) + rbrack + ofsy + <type>	array[1..10] of integer array[char] of integer
11	<range>	<constant> + period + period + <constant>	1..10, 'a'..'z'
12	<enumerated>	lparent + ident + (comma + ident) [*] + rparent	(Jan, Feb, Mar, Apr)
13	<record-type>	recordsy + <field-list> + endsy	type record-name = record field-1: field-type1; field-2: field-type2; ... field-n: field-typen; end

14	<field-list>	<field-part> + (semicolon + <field-part>)*	..field-1.. ..field-2..
15	<field-part>	<identifier-list> + colon + <type>	field-1: field-type1;
16	<subprogram-declaration>	<procedure-declaration> <function-declaration>	procedure print(x: integer);
17	<procedure-declaration>	proceduresy + ident + (<formal-parameter-list>)? + semicolon + <block> + semicolon	procedure swap(var a: integer; var b: integer); var temp: integer; begin temp := a; a := b; b := temp; end;
18	<function-declaration>	functionsy + ident + (<formal-parameter-list>)? + colon + ident + semicolon + <block> + semicolon	function tambah(a, b: integer): integer; begin tambah := a + b; end;
19	<block>	<declaration-part> + <compound-statement>	var ... begin ... end
20	<formal-parameter-list>	lparent + <parameter-group> + (semicolon + <parameter-group>)* + rparent	(x, y: integer; z: real)
21	<parameter-group>	<identifier-list> + colon + (ident <array-type>)	x, y: integer
22	<compound-statement>	beginsy + <statement-list>	begin ...

	nt>	+ endsy	end
23	<statement-list>	<statement> (semicolon + <statement>)*	begin if x > 0 then writeln('posit if') else writeln('negat if'); i := i + 1; for k := 1 to 10 do sum := sum + k end
24	<statement>	(<assignment-statement> <if-statement> <case-statement> <while-statement> <repeat-statement> <for-statement> <procedure/function-call>) ?	x := 5; if x > 0 then y := 1 else y := 0 while x < 10 do x := x + 1
25	<variable>	ident + (<component-variable>)*	someVar someArray[1] someField
26	<component-variable>	(lbrack + <index-list> + rbrack) (period + ident)	someArray[1] someRecord.som eField someArray[1][2]
27	<index-list>	(intcon charcon ident) + (comma + <index-list>)*	[1, 2, 3] ['a', 'b'] [true, false]
28	<assignment-statement>	<variable> + becomes + <expression>	x := 5, y := x + 10
29	<if-statement>	ifsy + <expression> + thensy + <statement> + (elsy + <statement>)?	if x > 0 then y := 1 else y := 0

30	<case-statement>	casesy + <expression> + ofsy + <case-block> + endsy	ease i of 0: x := 0; 1 : x := 1; end
31	<case-block>	<constant> + (comma + <constant>)* + colon + <statement> + (semicolon + <case-block?>)*	0: x := 0; 1, 2: x := 1
32	<while-statement>	whilesy + <expression> + dosy + <statement>	while x < 10 do x := x + 1
33	<repeat-statement> >	repeatsy + <statement-list> + untilsy + <expression>	repeat x := x - 1 until x == 0
34	<for-statement>	forsy + ident + becomes + <expression> + (tosy downtosy) + <expression> + dosy + <statement>	for i := 1 to 10 do ...
35	<procedure/function-call>	ident + (lparent + <parameter-list?> + rparent)	writeln('Hello '), print(x, y)
36	<parameter-list>	<expression>(comma + <expression>)*	'Result = ', b, 100
37	<expression>	<simple-expression>(<relational-operator> + <simple-expression>)?	x + y, a > b, 5 * (x + 1)
38	<simple-expression>	(plus minus)? <term> (<additive-operator> + <term>)*	x + y - z, 5 * 2
39	<term>	<factor>(<multiplicative-operator> + <factor>)*	x * y, a div b
40	<factor>	ident intcon realcon charcon string (lparent + <expression> + rparent) (notsy + <factor>) <procedure/function-call> <variable>	x, 5, 'a', (x+y), not a
41	<relational-operator>	eq neq gtw geq lss leq	x == 5, y > 10

42	<additive-operator>	plus minus orsy	x + y, a or b
43	<multiplicative-operator>	times rdiv idiv imod andsy	x * y, a div b, p and q

BAB III

PENGUJIAN

3.1 Test Case 1

Test case valid, semua aturan produksi terpenuhi dengan baik.

INPUT

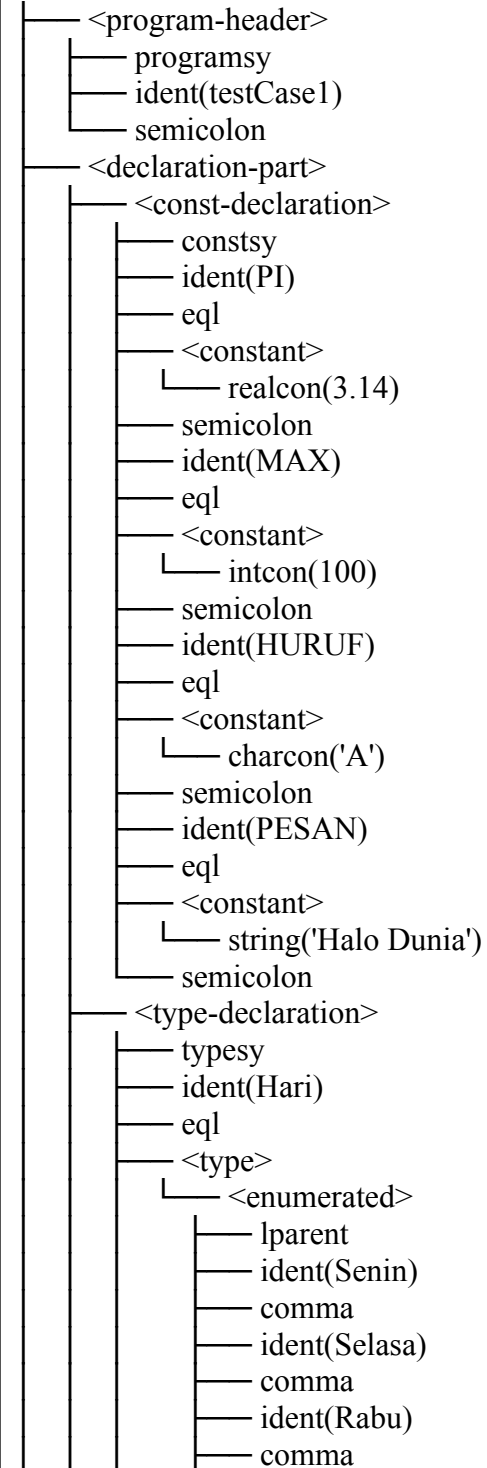
```
test > milestone-2 > testCase1.txt
1  program testCase1;
2
3  const
4      PI == 3.14;
5      MAX == 100;
6      HURUF == 'A';
7      PESAN == 'Halo Dunia';
8
9  type
10     Hari == (Senin, Selasa, Rabu, Kamis, Jumat);
11     Batas == 1 .. 100;
12     Siswa == record
13         umur : integer;
14         nilai : real;
15         lulus : boolean
16     end;
17     DataSiswa == array [1..100] of Siswa;
18     Matriks == array [1..10] of array [1..10] of integer;
19
20  var
21     i, j, k : integer;
22     murid : Siswa;
23     kelas : DataSiswa;
24     papan : Matriks;
25     libur : Hari;
26
27  begin
28     i := +10;
29     j := -5;
30
31     murid.umur := 15;
32     murid.nilai := PI * 2.0;
33     kelas[1].lulus := true;
34     papan[1][2] := 100;
35     libur := Senin;
36
37     case i of
38         1, 2, 3 : k := 10;
39         4 : k := 20;
40         5 : k := 0
41     end;
42
43     for i := 1 to MAX do
44         if i == 50 then
45             kelas[i].lulus := true
46         else
47             kelas[i].lulus := false;
48     end.
```

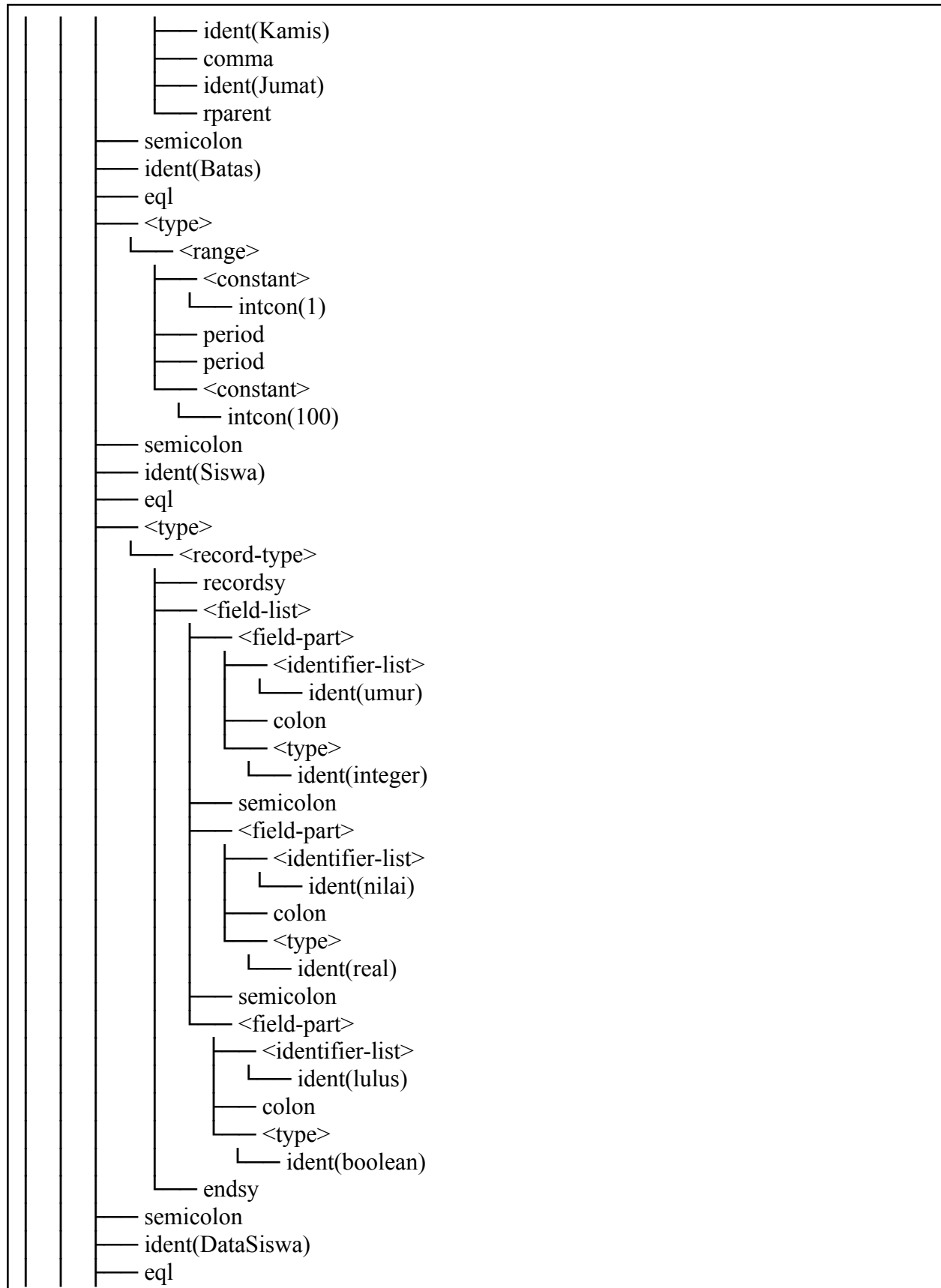
OUTPUT

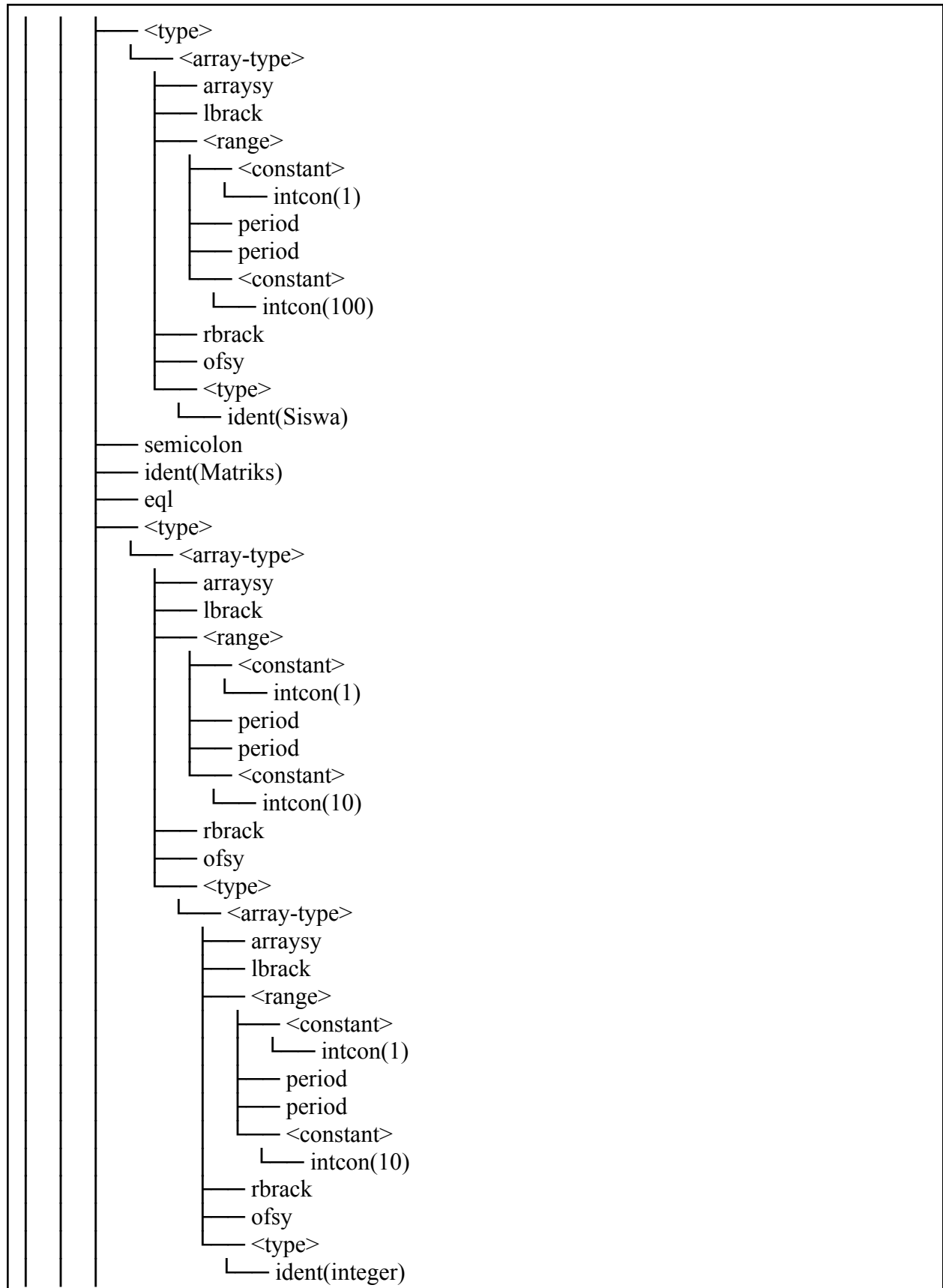
=== TIDAK ADA SYNTAX ERROR (PROGRAM VALID) ===

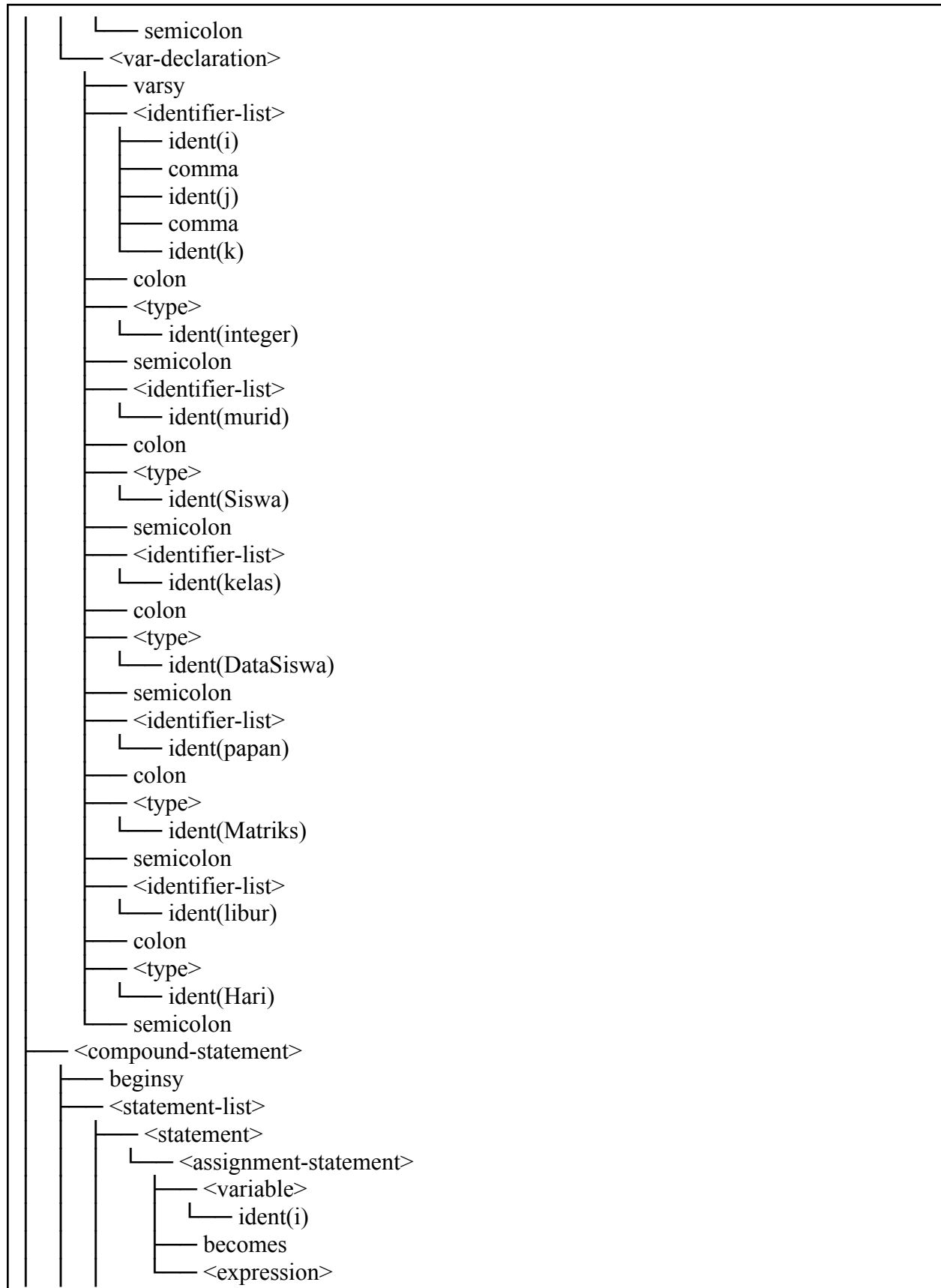
=== HASIL PARSE TREE ===

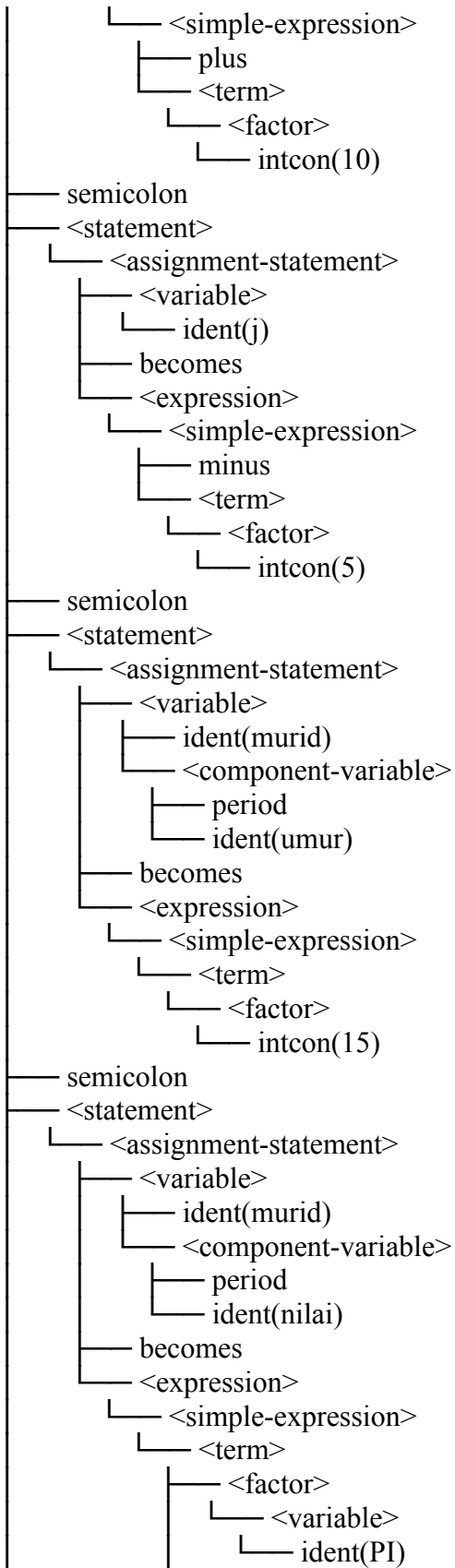
<program>

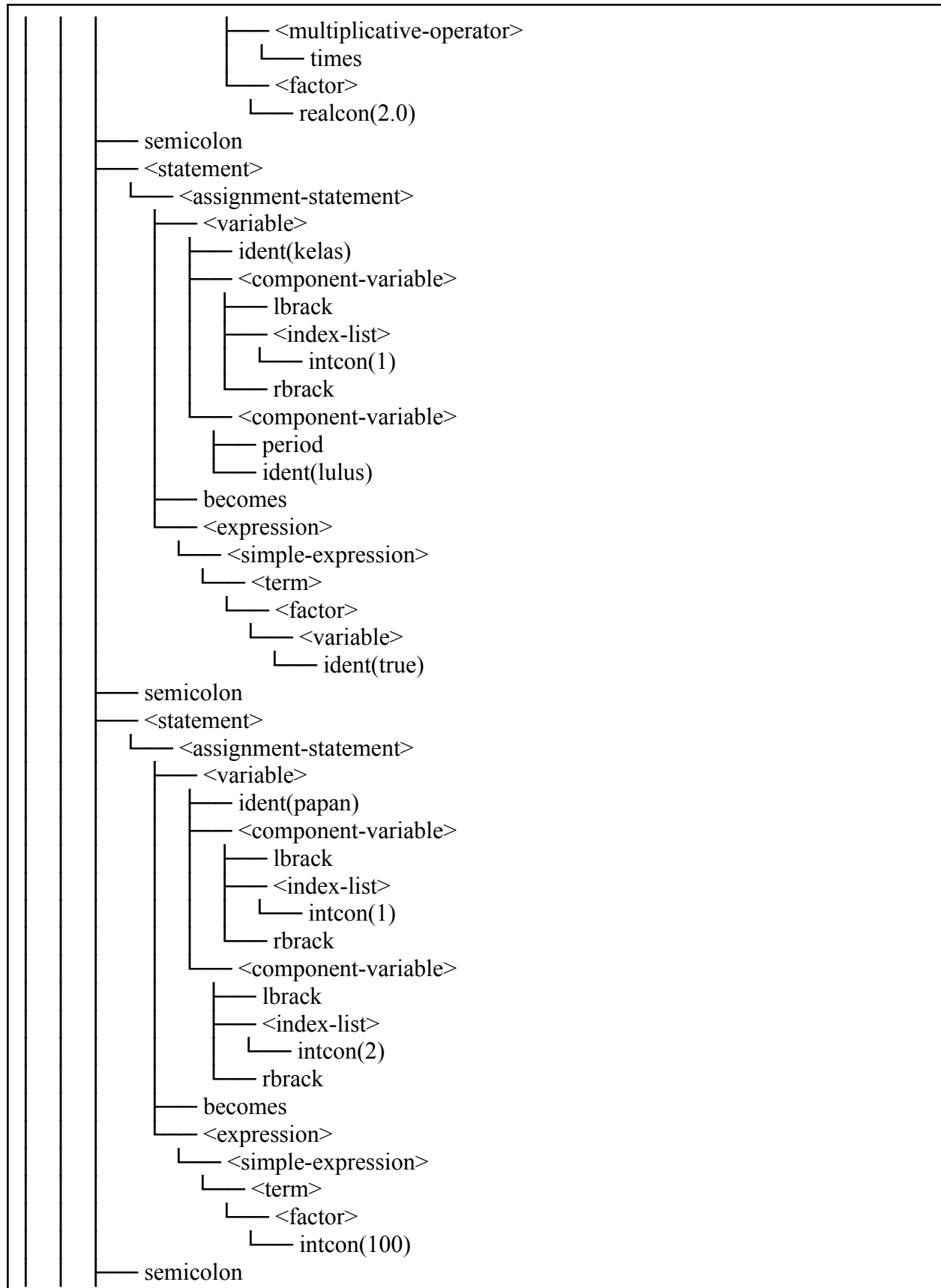


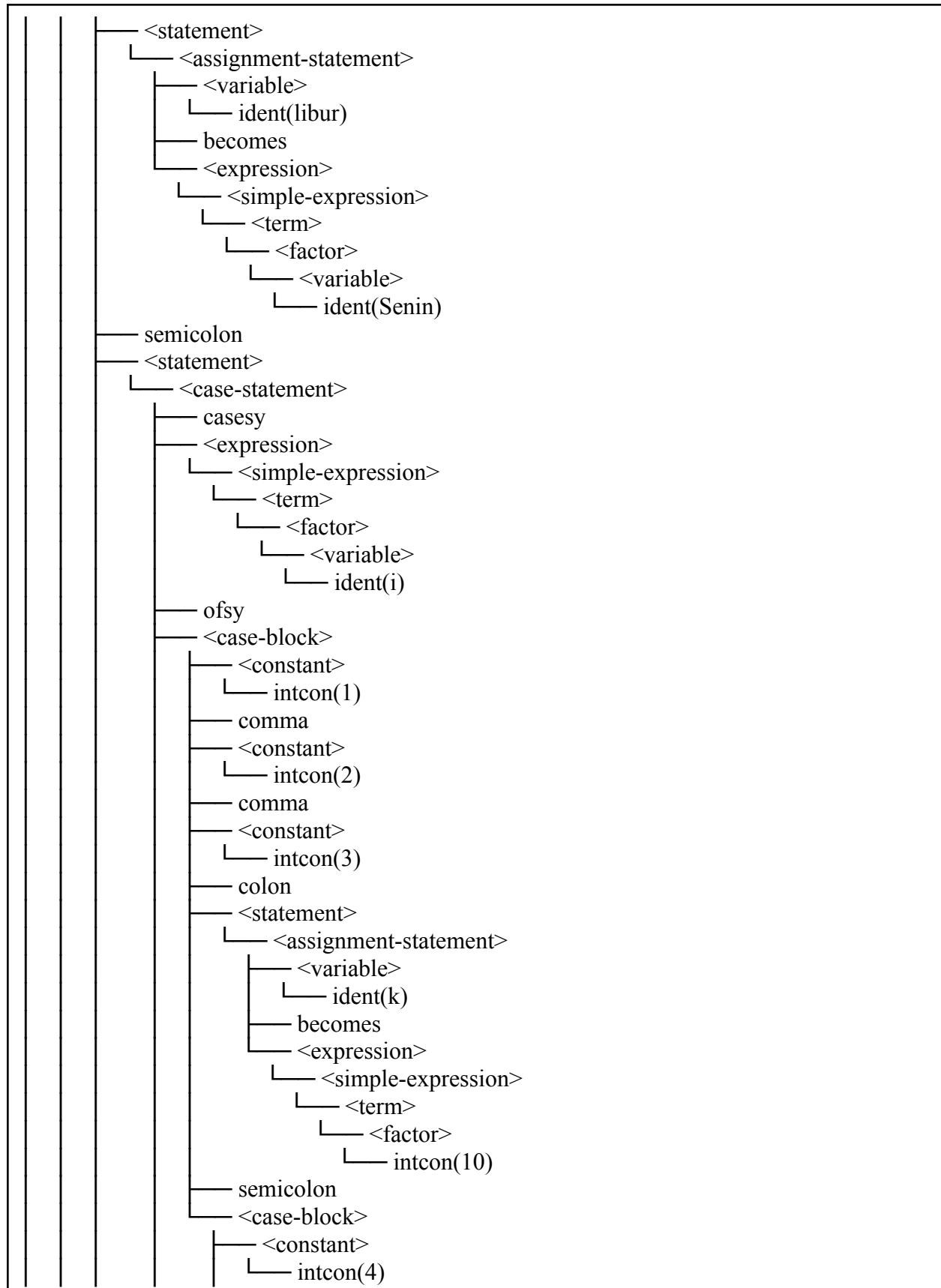


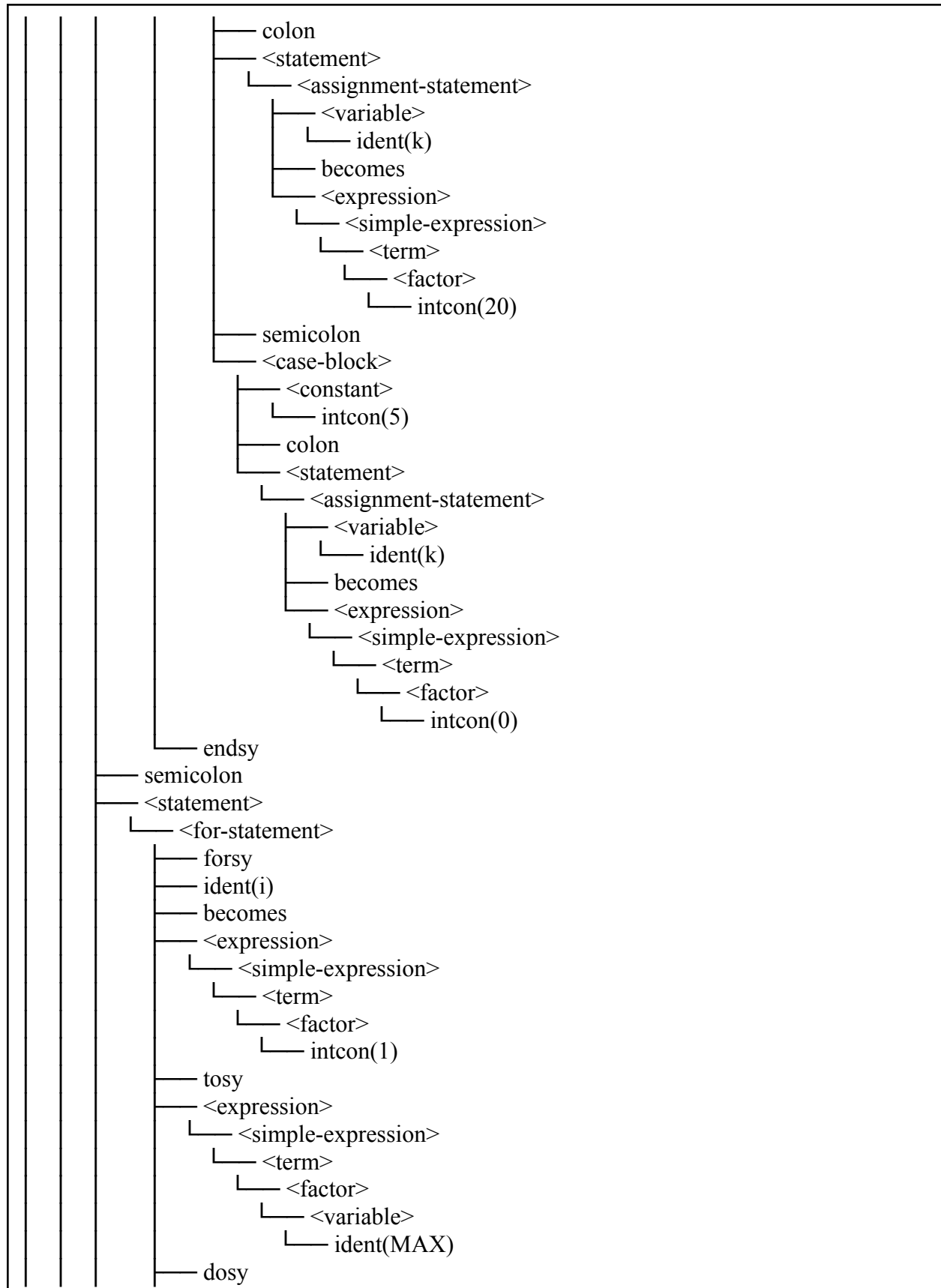


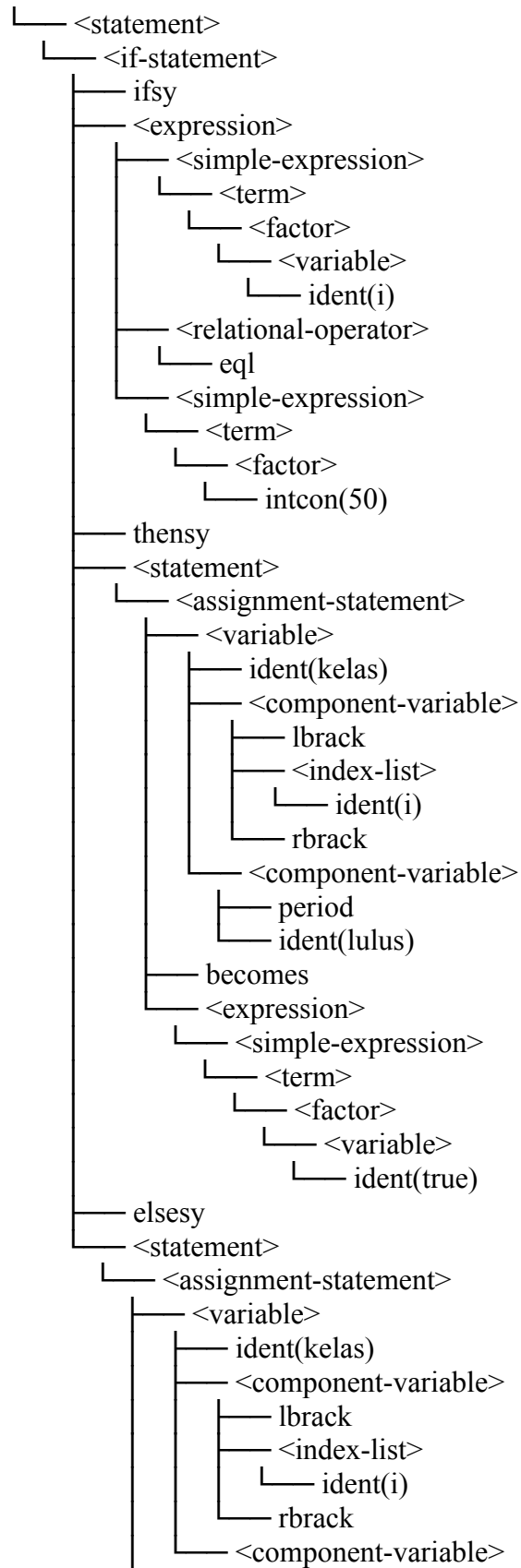


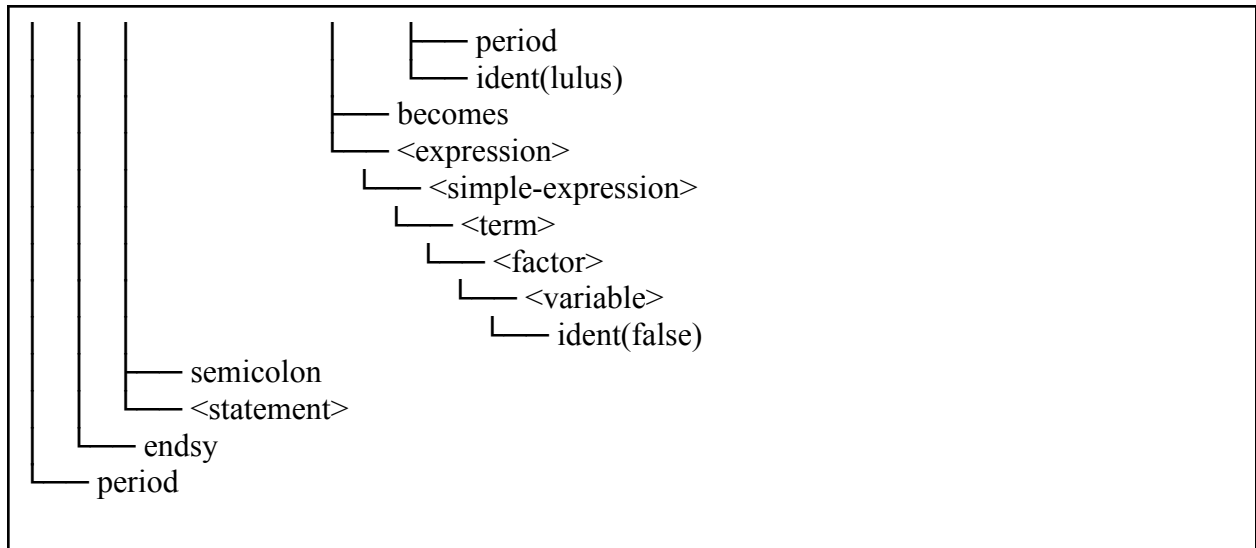












3.2 Test Case 2

Test case valid, semua aturan produksi terpenuhi dengan baik.

INPUT

```

1  program testCase2;
2
3  var
4      x, y, z : integer;
5      status : boolean;
6      hasil : real;
7
8  procedure HitungMundur(start : integer; prefix : string);
9  var
10     counter : integer;
11 begin
12     counter := start;
13     while counter > 0 do
14         counter := counter - 1;
15 end;
16
17 function CekValid(a, b : integer; limit : real) : boolean;
18 var
19     temp : real;
20 begin
21     temp := (a + b) / 2.0;
22     if temp <= limit then
23         CekValid := true
24     else
25         CekValid := false;
26 end;
27
28 begin
29     x := 10;
30     y := 20;
31
32     HitungMundur(x, 'Mulai: ');
33
34     status := not CekValid(x, y, 15.5) and (x <> y) or (x == 10);
35
36     repeat
37         z := x * y div 2 mod 3;
38         x := x + 1;
39     until x >= 20;
40
41     for z := 10 downto 1 do
42         y := y - 1;
43 end.

```

OUTPUT

=== TIDAK ADA SYNTAX ERROR (PROGRAM VALID) ===

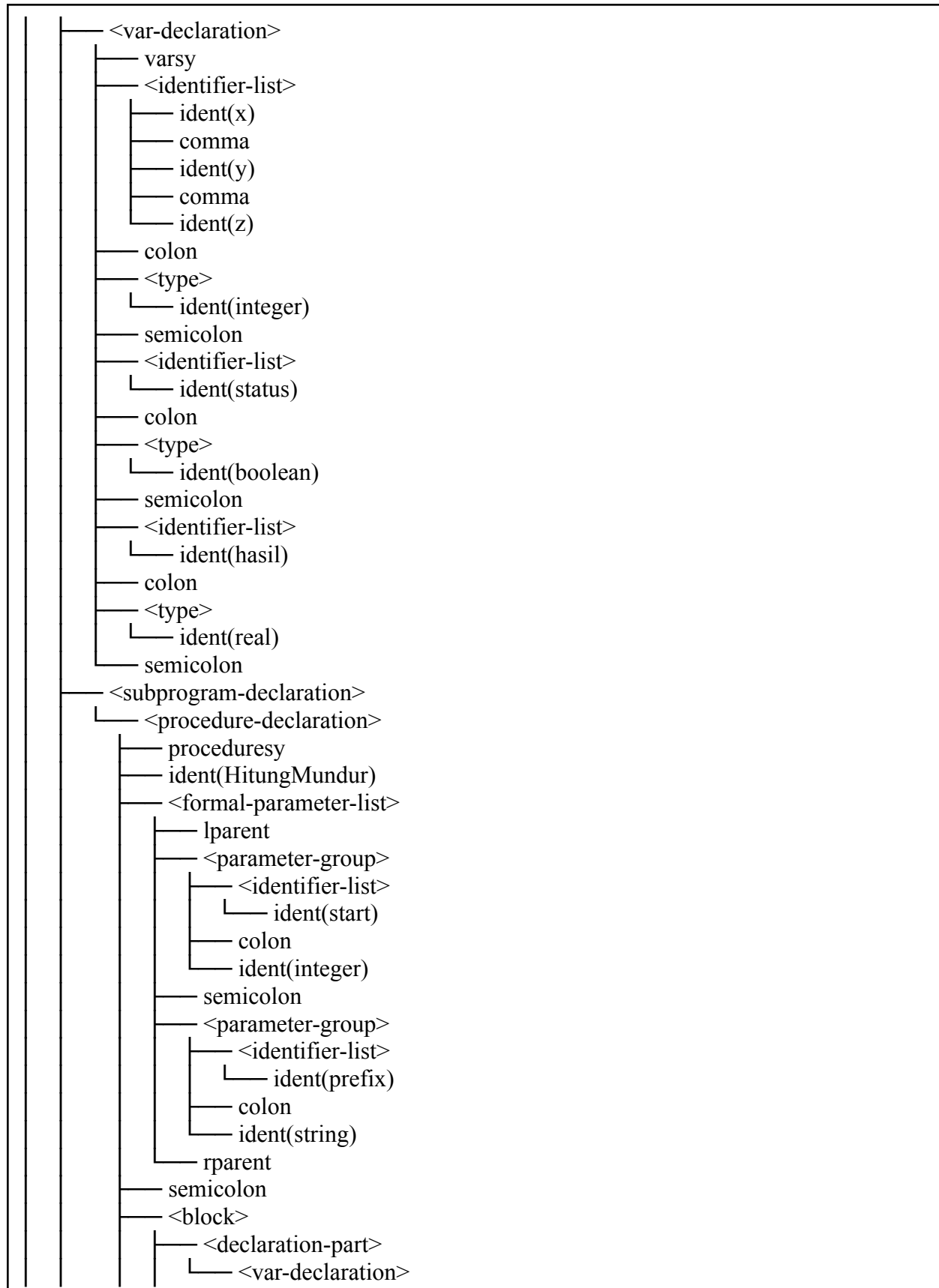
=== HASIL PARSE TREE ===

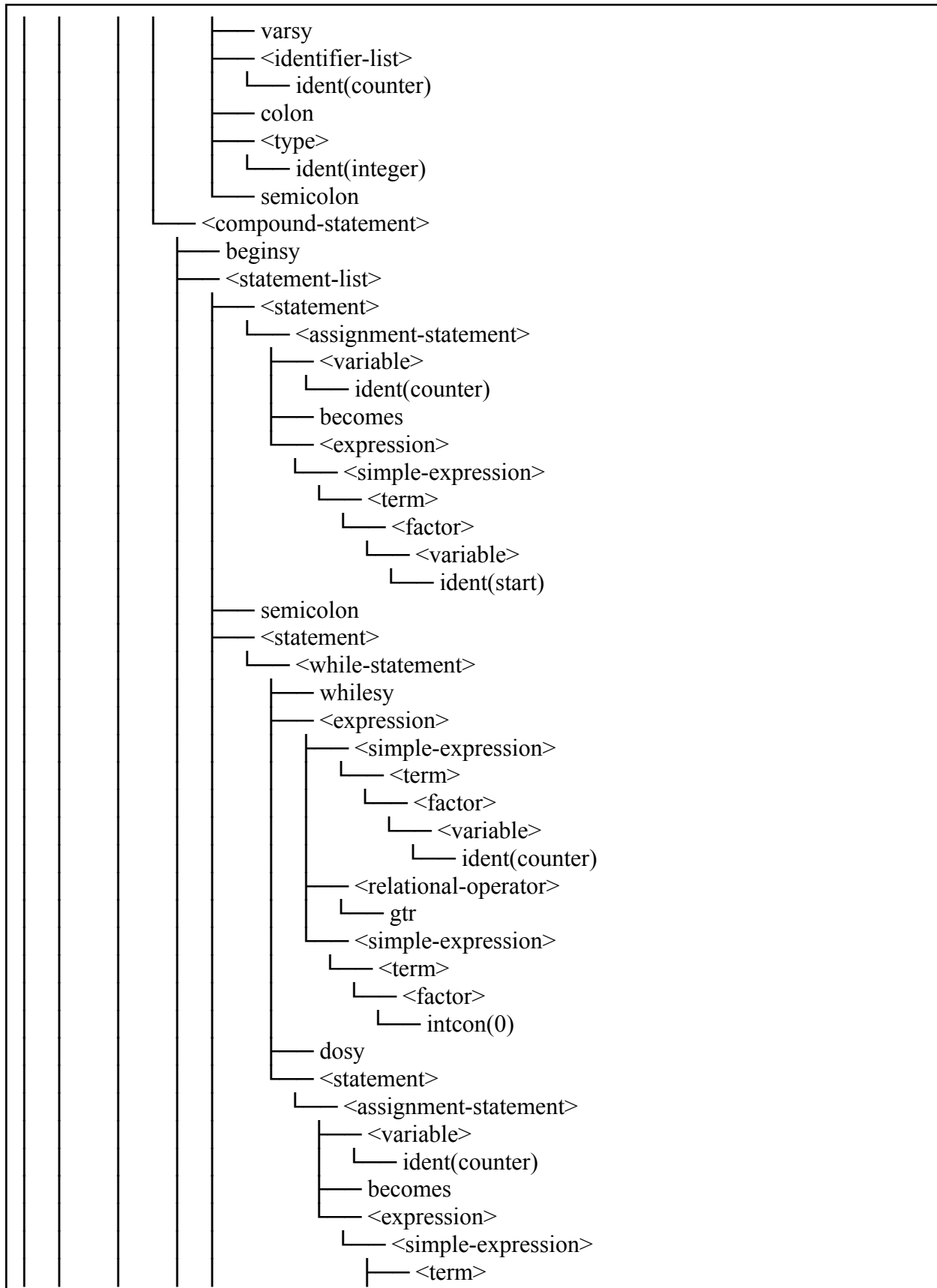
<program>

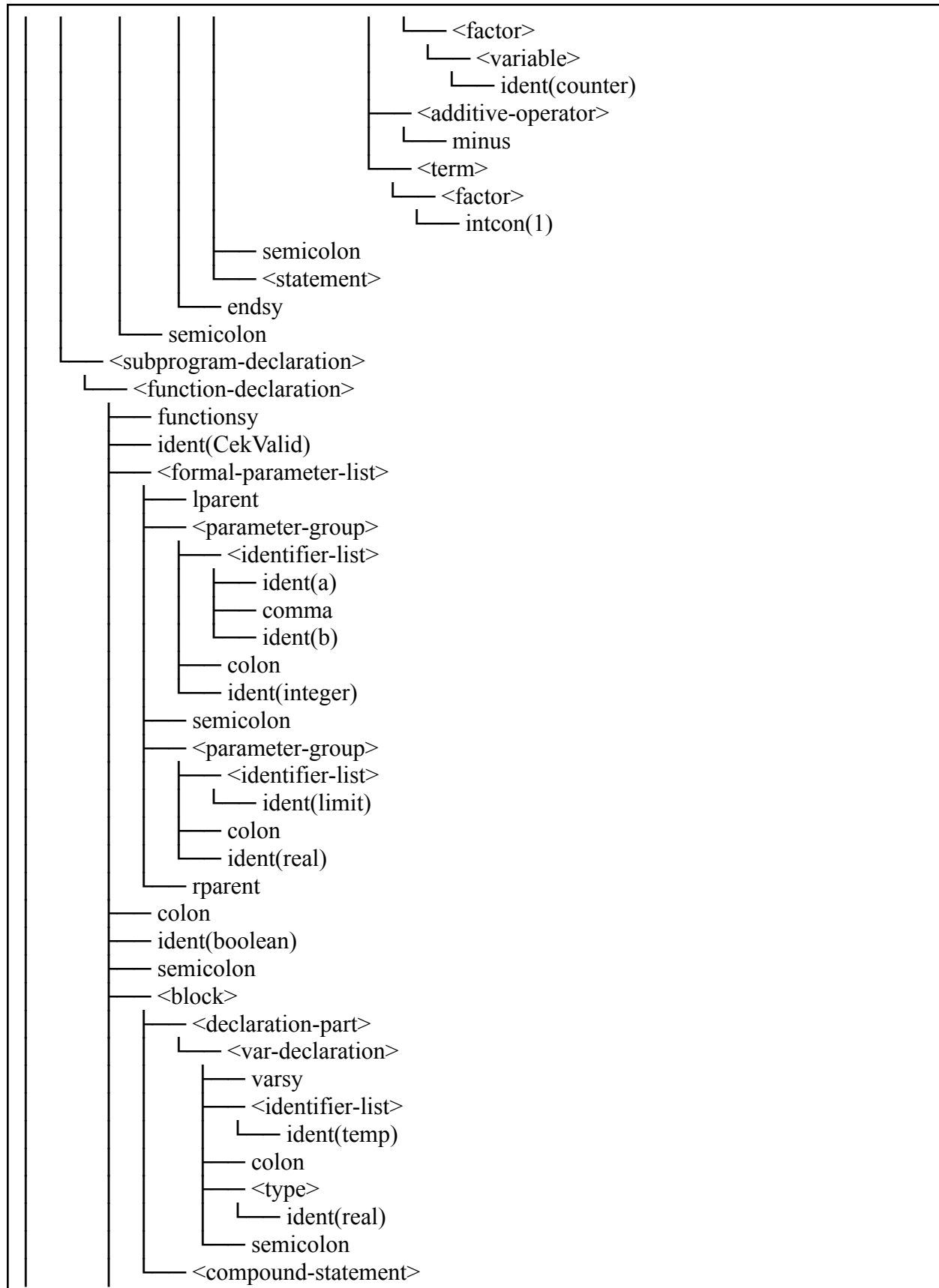
```

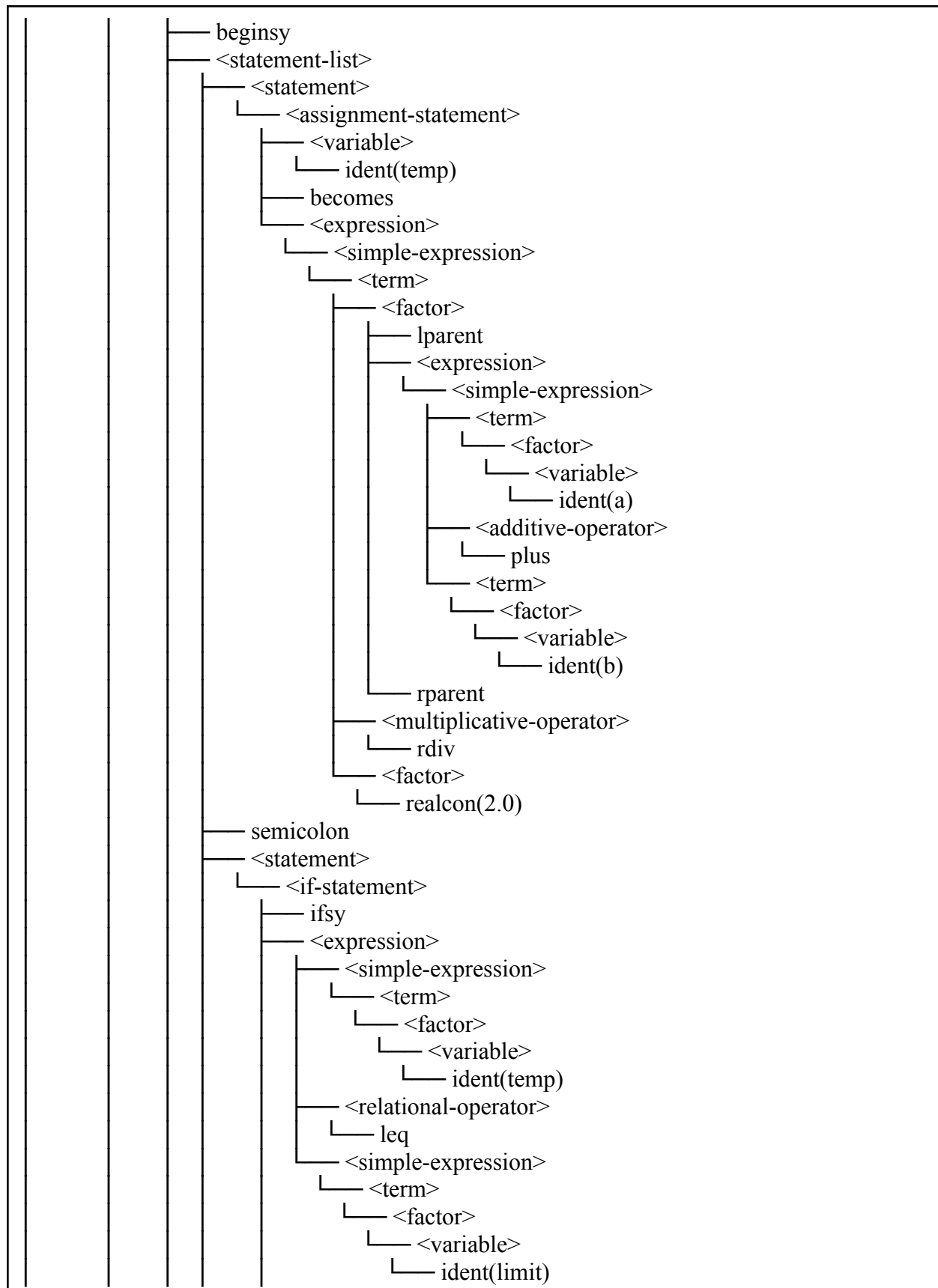
├── <program-header>
│   ├── programsy
│   ├── ident(testCase2)
│   └── semicolon
└── <declaration-part>

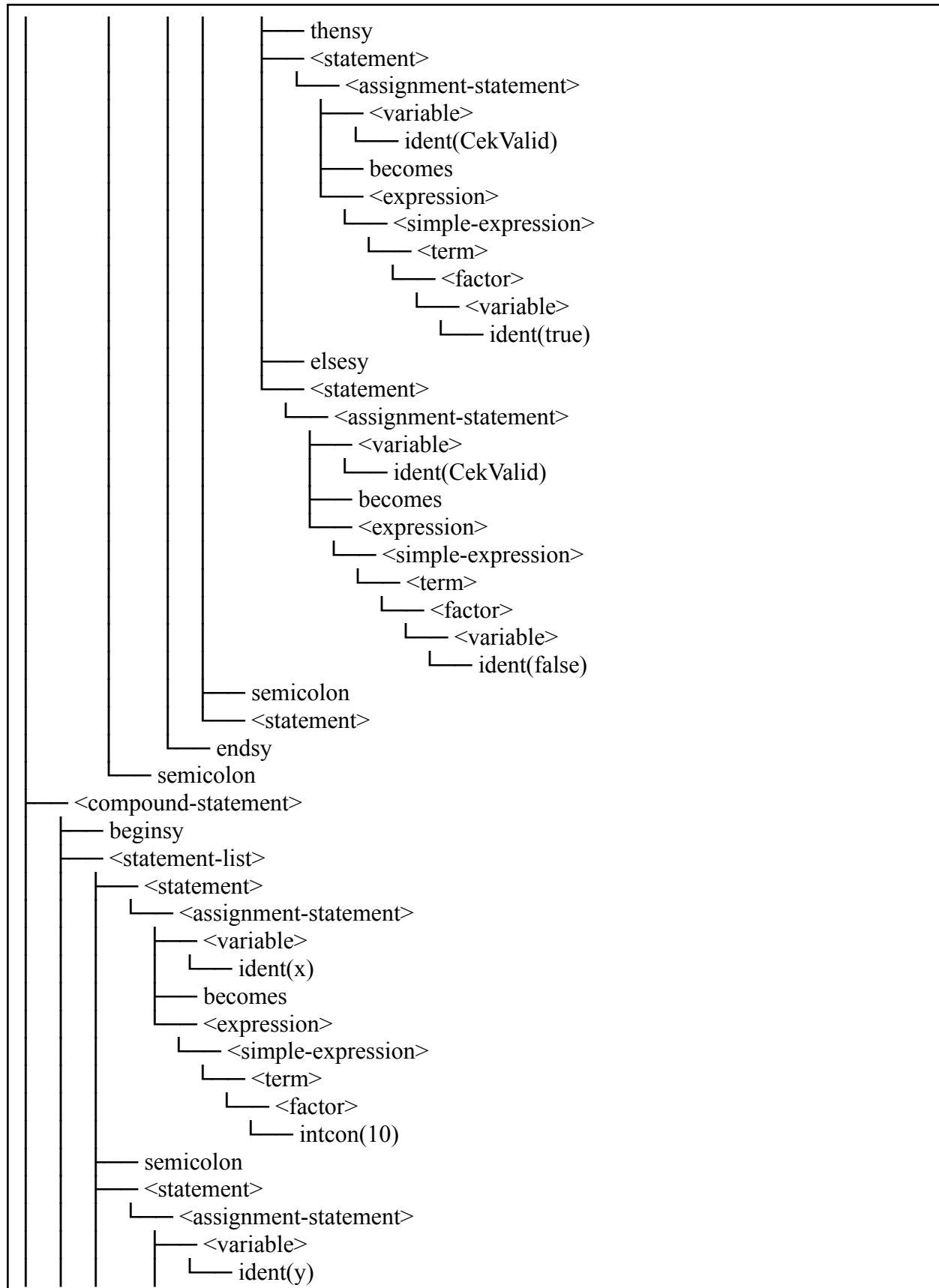
```

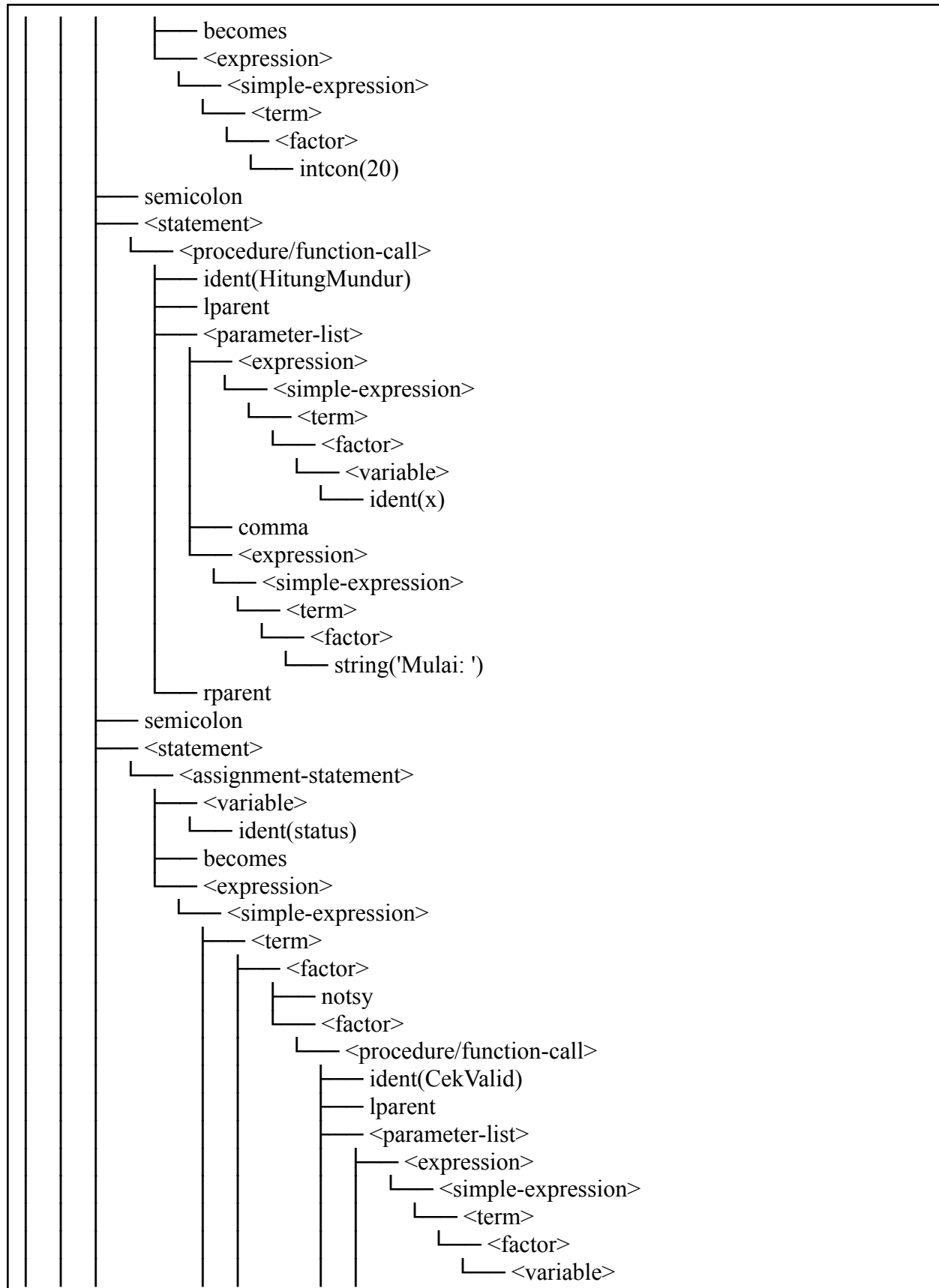


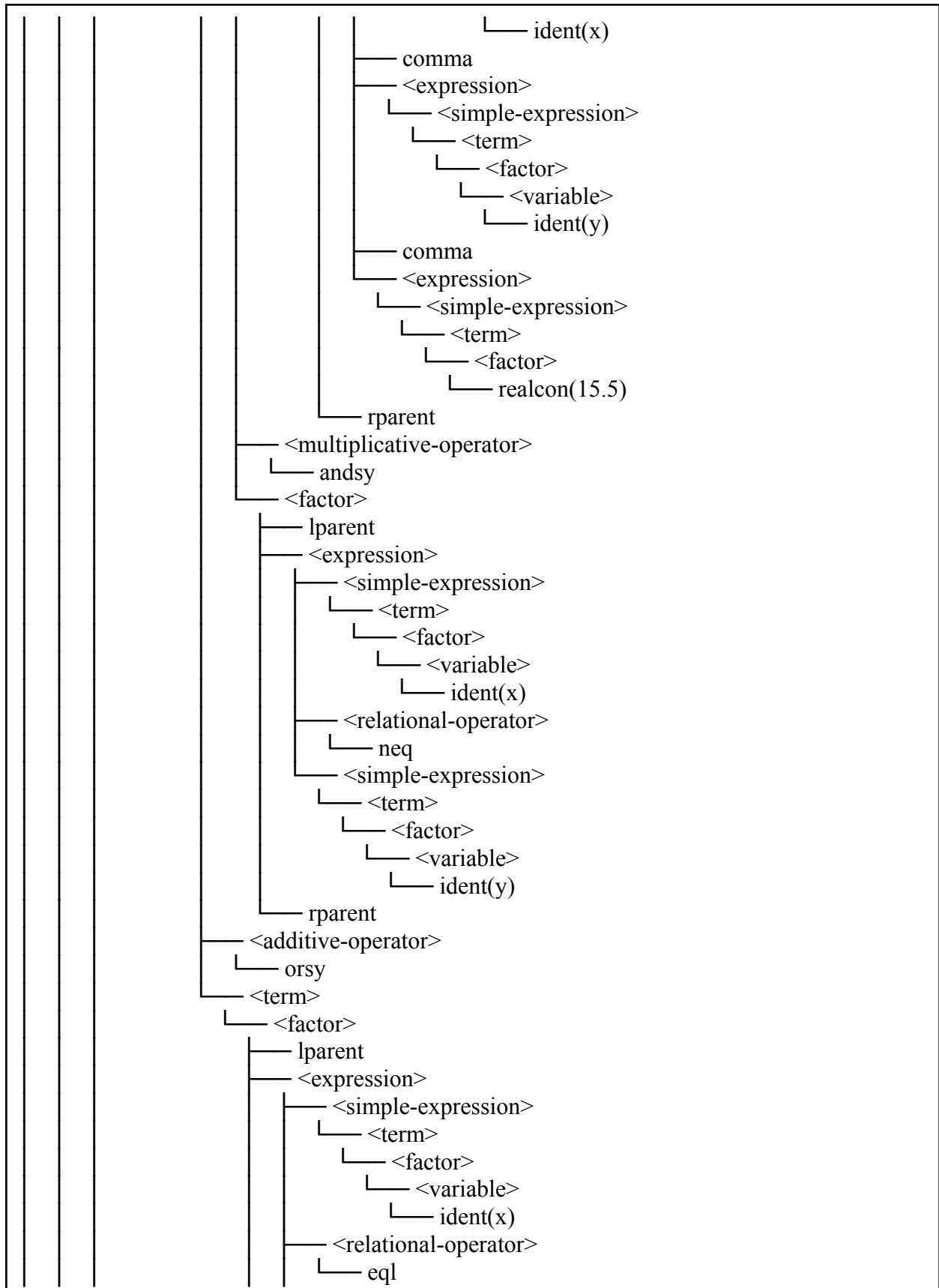


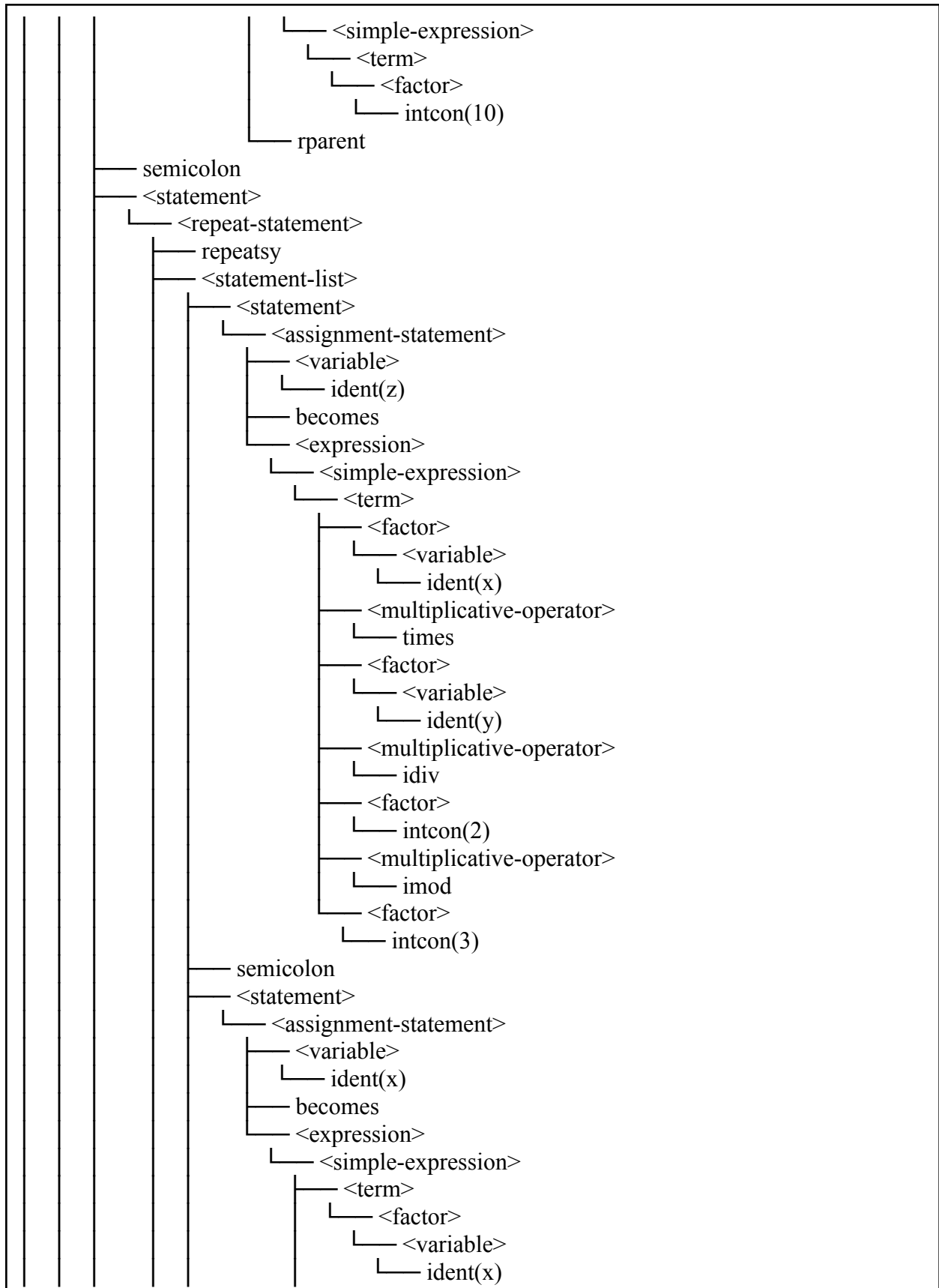


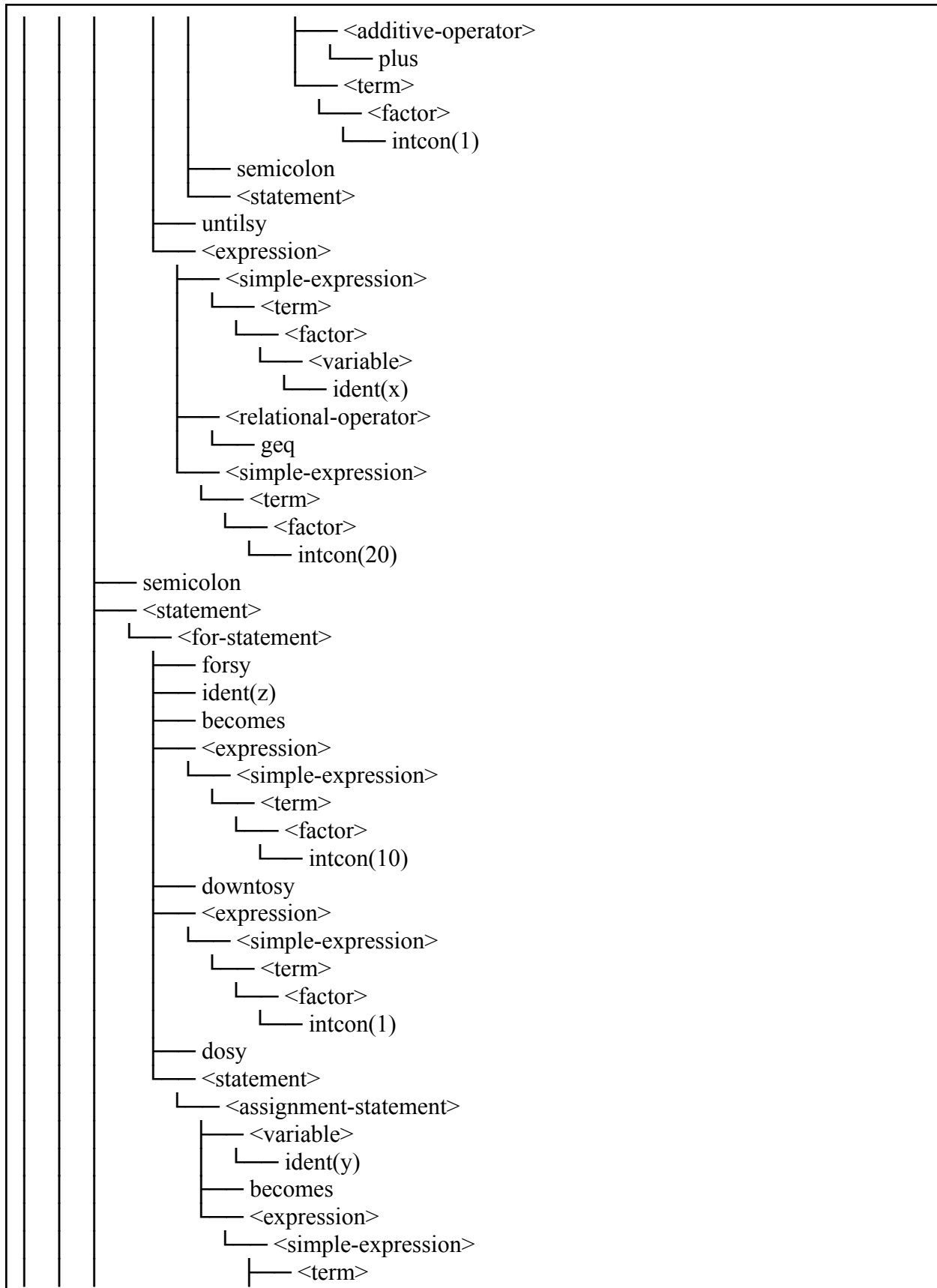


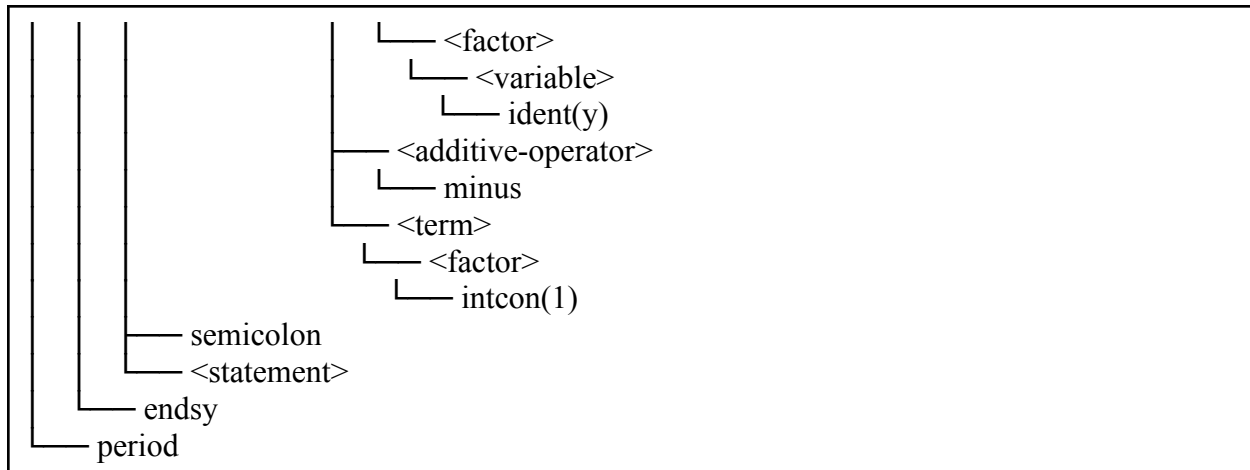












3.3 Test Case 3

Test Case gagal karena setelah ':' diharapkan *identifier* seperti integer atau string.

INPUT

```

test > milestone-2 > testCase3.txt
1  program testCase3;
2
3  var
4  |  angka := ; { ERROR: Seharusnya menuliskan tipe data seperti 'integer' }
5
6  begin
7  |  angka := 10;
8  end.
  
```

OUTPUT

```

test > milestone-2 > testCase3-Result-Parse.txt
1  === DAFTAR SYNTAX ERROR ===
2  [SYNTAX ERROR] Mengharapkan 'ident' tetapi menemukan ':' pada baris 4!
3
4  === HASIL PARSE TREE ===
5  <program>
6  |  <program-header>
7  |  |  programsy
8  |  |  ident(testCase3)
9  |  |  semicolon
10 |  <declaration-part>
11 |  |  <var-declaration>
12 |  |  |  varsy
13 |  |  |  <identifier-list>
14 |  |  |  |  ident(angka)
15 |  |  |  colon
16 |  |  |  <type>
17 |  |  |  |  ERROR_STOP
18 |  |  |  SKIPPED
19 |  <compound-statement>
20 |  SKIPPED
  
```

3.4 Test Case 4

Test Case gagal karena *for loop* di ARION membutuhkan 'to' setelah inisialisasi i.

Seharusnya menjadi :

for i := 1 to 10 do

INPUT

```
test > milestone-2 > testCase4.txt
1  program testCase4;
2
3  var
4  |   i, total : integer;
5
6  begin
7  |   total := 0;
8
9  |   { ERROR: Lupa menuliskan 'to' di antara angka 1 dan 10 }
10 |   for i := 1 10 do
11 |       total := total + i;
12 end.
```

OUTPUT

```

1  === DAFTAR SYNTAX ERROR ===
2  [SYNTAX ERROR] Mengharapkan 'tosy' tetapi menemukan '10' pada baris 10!
3
4  === HASIL PARSE TREE ===
5  <program>
6  |
7  |--- <program-header>
8  |   |
9  |   |--- programsy
10  |   |--- ident(testCase4)
11  |   |--- semicolon
12  |
13  |--- <declaration-part>
14  |   |
15  |   |--- <var-declaration>
16  |   |   |
17  |   |   |--- varsy
18  |   |   |   |
19  |   |   |   |--- <identifier-list>
20  |   |   |   |   |
21  |   |   |   |   |--- ident(i)
22  |   |   |   |   |--- comma
23  |   |   |   |   |--- ident(total)
24  |   |   |   |   |--- colon
25  |   |   |   |   |--- <type>
26  |   |   |   |   |   |--- ident(integer)
27  |   |   |   |   |--- semicolon
28  |   |
29  |   |--- <compound-statement>
30  |   |   |
31  |   |   |--- beginsy
32  |   |   |   |
33  |   |   |   |--- <statement-list>
34  |   |   |   |   |
35  |   |   |   |   |--- <statement>
36  |   |   |   |   |   |
37  |   |   |   |   |   |--- <assignment-statement>
38  |   |   |   |   |   |   |
39  |   |   |   |   |   |   |--- <variable>
40  |   |   |   |   |   |   |   |
41  |   |   |   |   |   |   |   |--- ident(total)
42  |   |   |   |   |   |   |   |--- becomes
43  |   |   |   |   |   |   |   |--- <expression>
44  |   |   |   |   |   |   |   |   |
45  |   |   |   |   |   |   |   |   |--- <simple-expression>
46  |   |   |   |   |   |   |   |   |   |
47  |   |   |   |   |   |   |   |   |   |--- <term>
48  |   |   |   |   |   |   |   |   |   |   |
49  |   |   |   |   |   |   |   |   |   |   |--- <factor>
50  |   |   |   |   |   |   |   |   |   |   |   |--- intcon(0)
51  |   |   |   |   |   |   |   |   |   |--- semicolon
52  |   |   |   |   |   |   |   |--- <statement>
53  |   |   |   |   |   |   |   |   |
54  |   |   |   |   |   |   |   |   |--- <for-statement>
55  |   |   |   |   |   |   |   |   |   |
56  |   |   |   |   |   |   |   |   |   |--- forsy
57  |   |   |   |   |   |   |   |   |   |--- ident(i)
58  |   |   |   |   |   |   |   |   |   |--- becomes
59  |   |   |   |   |   |   |   |   |   |--- <expression>
60  |   |   |   |   |   |   |   |   |   |   |
61  |   |   |   |   |   |   |   |   |   |   |--- <simple-expression>
62  |   |   |   |   |   |   |   |   |   |   |   |
63  |   |   |   |   |   |   |   |   |   |   |   |--- <term>
64  |   |   |   |   |   |   |   |   |   |   |   |   |
65  |   |   |   |   |   |   |   |   |   |   |   |   |--- <factor>
66  |   |   |   |   |   |   |   |   |   |   |   |   |   |--- intcon(1)
67  |   |   |   |   |   |   |   |   |   |--- ERROR_STOP
68  |   |   |   |   |   |   |   |   |   |--- <expression>
69  |   |   |   |   |   |   |   |   |   |--- SKIPPED
70  |   |   |   |   |   |   |   |   |--- <statement>
71  |   |   |   |   |   |   |--- SKIPPED
72  |   |--- SKIPPED
73  |--- SKIPPED

```

3.5 Test Case 5

Test case gagal, karena program mengharapkan 'ident' tetapi menemukan '*'.

Seharusnya menjadi:

hasil := 10 + 5;

INPUT

```
test > milestone-2 > testCase5.txt
1  program testCase5;
2
3  var
4  |   hasil : integer;
5
6  begin
7  |   { ERROR: Ada dua operator bersebelahan (+ dan *) }
8  |   hasil := 10 + * 5;
9  end.
```

OUTPUT

```
test > milestone-2 > testCase5-Result-Parse.txt
1  === DAFTAR SYNTAX ERROR ===
2  [SYNTAX ERROR] Mengharapkan 'ident' tetapi menemukan '*' pada baris 8!
3
4  === HASIL PARSE TREE ===
5  <program>
6  |   <program-header>
7  |   |   programsy
8  |   |   ident(testCase5)
9  |   |   semicolon
10 |   <declaration-part>
11 |   |   <var-declaration>
12 |   |   |   varsy
13 |   |   |   <identifier-list>
14 |   |   |   |   ident(hasil)
15 |   |   |   colon
16 |   |   |   <type>
17 |   |   |   |   ident(integer)
18 |   |   |   semicolon
19 |   <compound-statement>
20 |   |   beginsy
21 |   |   <statement-list>
22 |   |   |   <statement>
23 |   |   |   |   <assignment-statement>
24 |   |   |   |   |   <variable>
25 |   |   |   |   |   |   ident(hasil)
26 |   |   |   |   |   becomes
27 |   |   |   |   |   <expression>
28 |   |   |   |   |   |   <simple-expression>
29 |   |   |   |   |   |   |   <term>
30 |   |   |   |   |   |   |   |   <factor>
31 |   |   |   |   |   |   |   |   |   intcon(10)
32 |   |   |   |   |   |   |   |   <additive-operator>
33 |   |   |   |   |   |   |   |   |   plus
34 |   |   |   |   |   |   |   |   <term>
35 |   |   |   |   |   |   |   |   |   <factor>
36 |   |   |   |   |   |   |   |   |   |   ERROR_STOP
37 |   |   |   |   |   |   |   |   SKIPPED
38 |   |   |   |   |   |   |   SKIPPED
```

3.6 Test Case 6

Test case valid, semua aturan produksi terpenuhi dengan baik.

INPUT

```
test > milestone-2 > testCase6.txt
1  { state skip: brace comment }
2  (*
3  state skip: paren-star comment
4  state skip: paren-star comment
5  state skip: paren-star comment
6  state skip: paren-star comment
7  state skip: paren-star comment
8  state skip: paren-star comment
9  *)
10
11  program p5;
12  const c1 == 123;
13  c2 == 123;
14  type t1 == hama;
15  var v1, v2, v3: array[1..3] of integer;
16
17  function f1(a1: integer): integer;
18  begin
19      if a1 >= 10 then
20          a1 := a1 - 1
21      else
22          a1 := a1 + 2;
23          f1 := a1 div 2 mod 3;
24  end;
25
26  procedure p1;
27  begin
28      repeat
29          while v1 < v2 do
30              v1 := v1 + 1;
31              for v3 := 10 downto 1 do
32                  v2 := v2 * 2 / 4;
33              until v1 == v2;
34  end;
35
36  begin
37      v1 := -42;
38      v2 := 3.1415;
39  end.
40
```

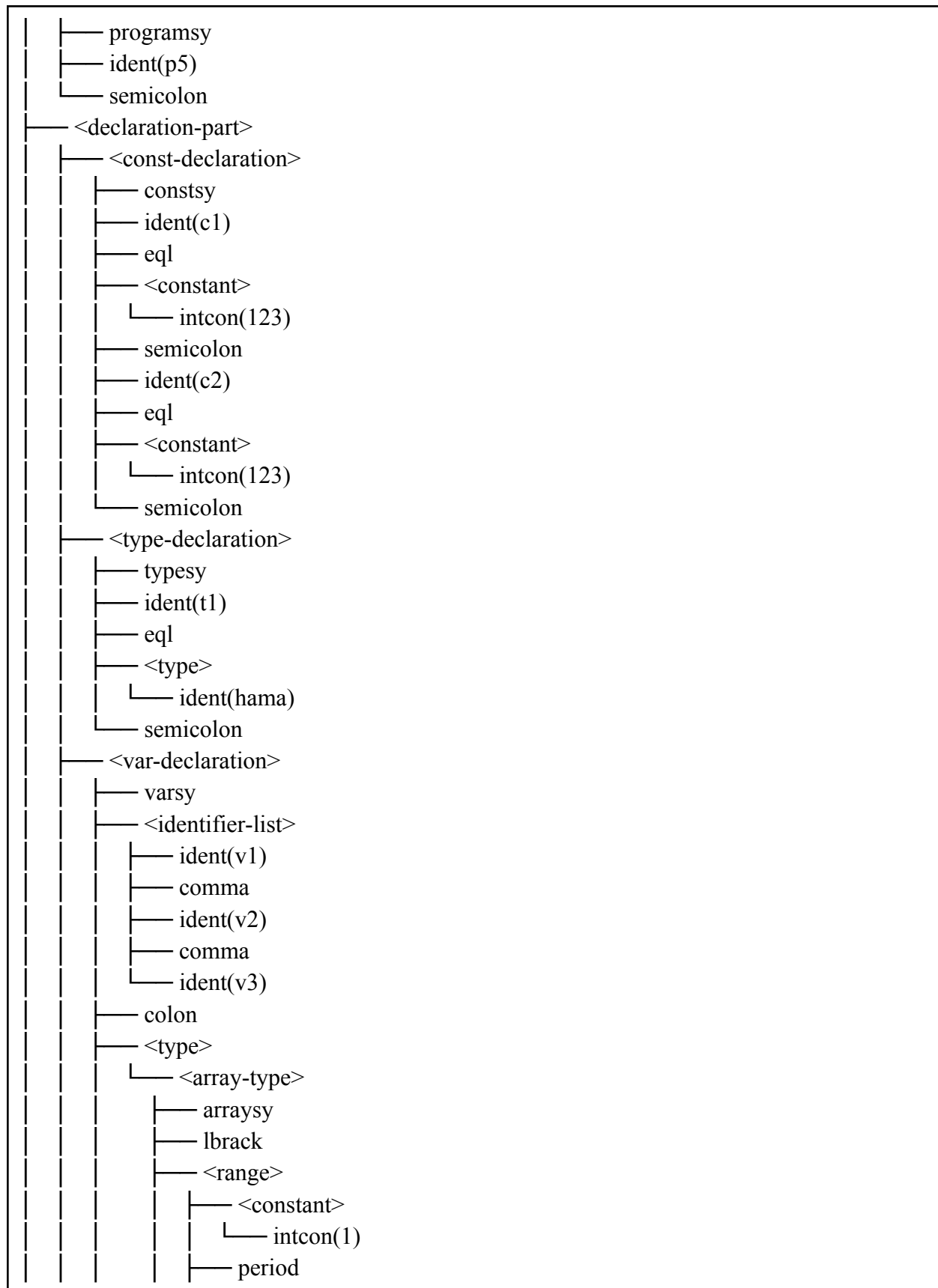
OUTPUT

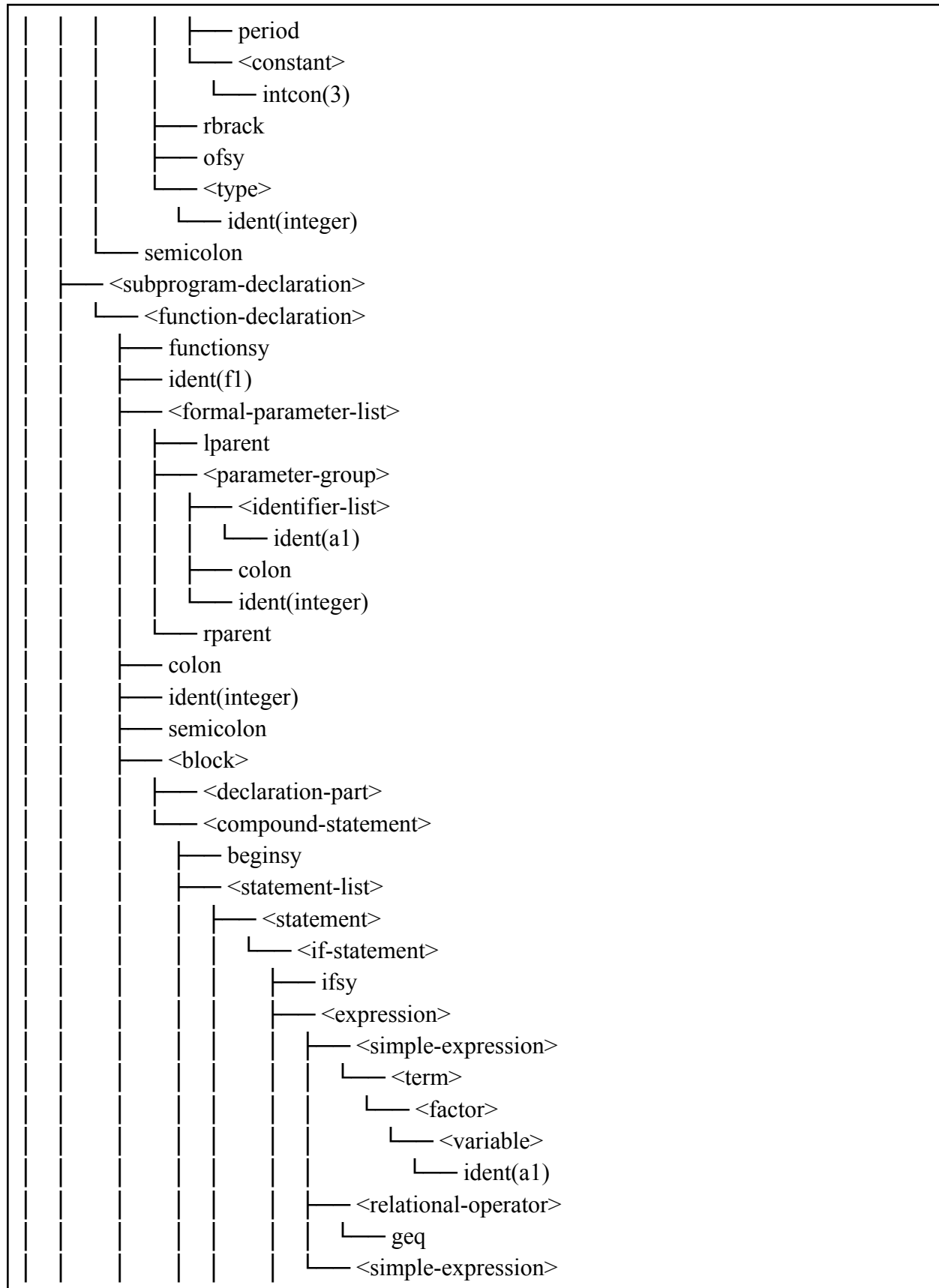
=== TIDAK ADA SYNTAX ERROR (PROGRAM VALID) ===

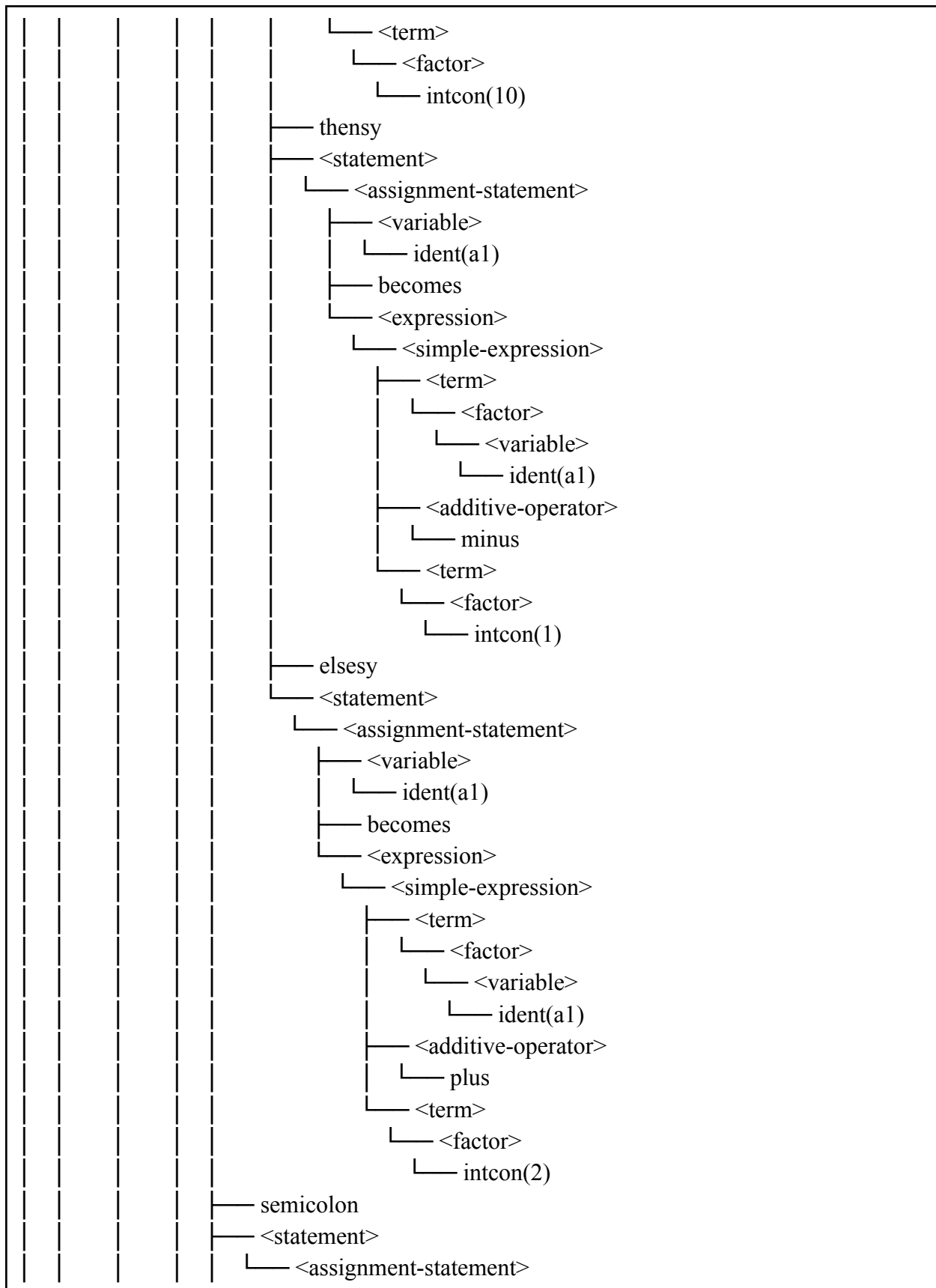
=== HASIL PARSE TREE ===

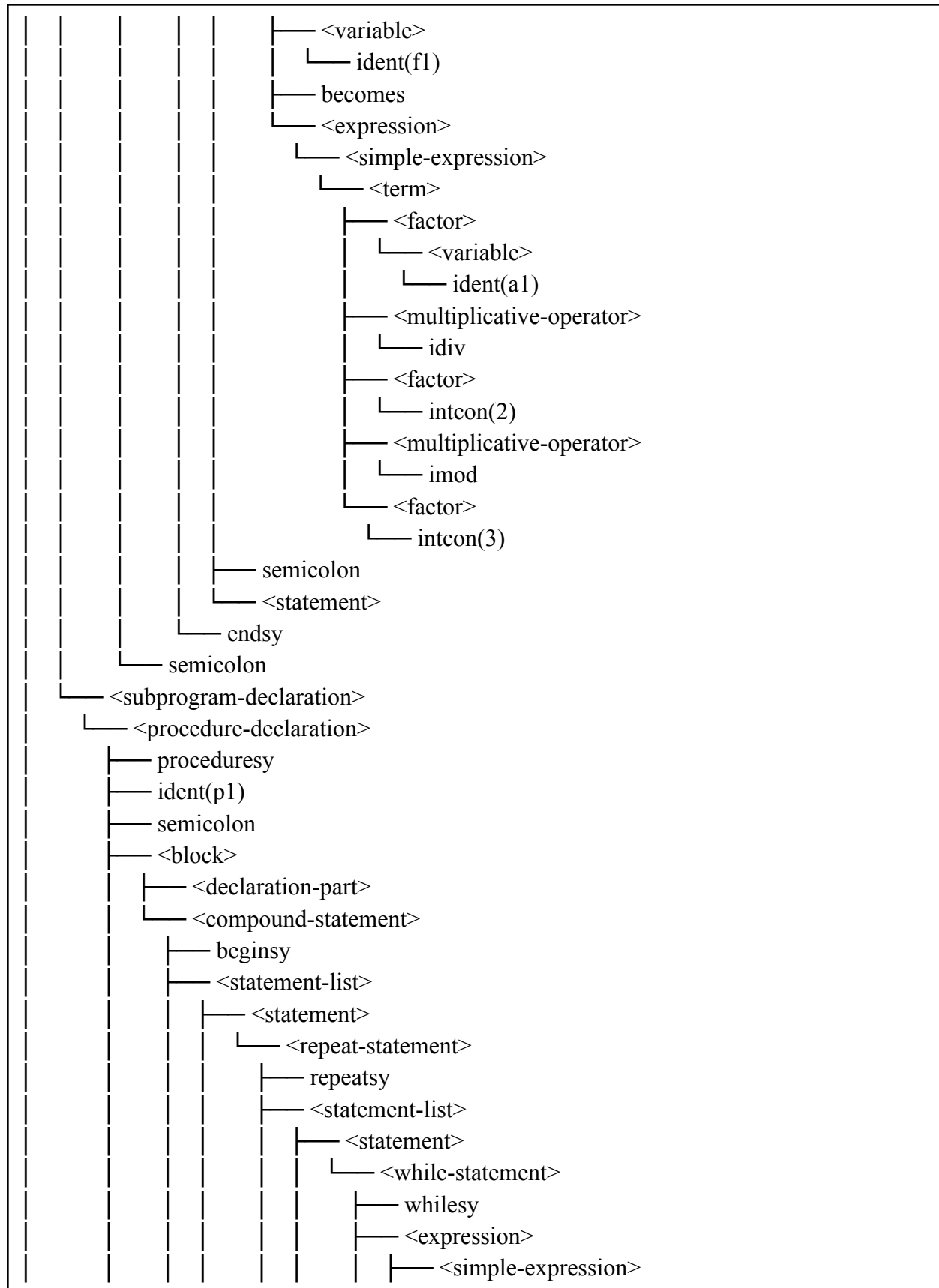
<program>

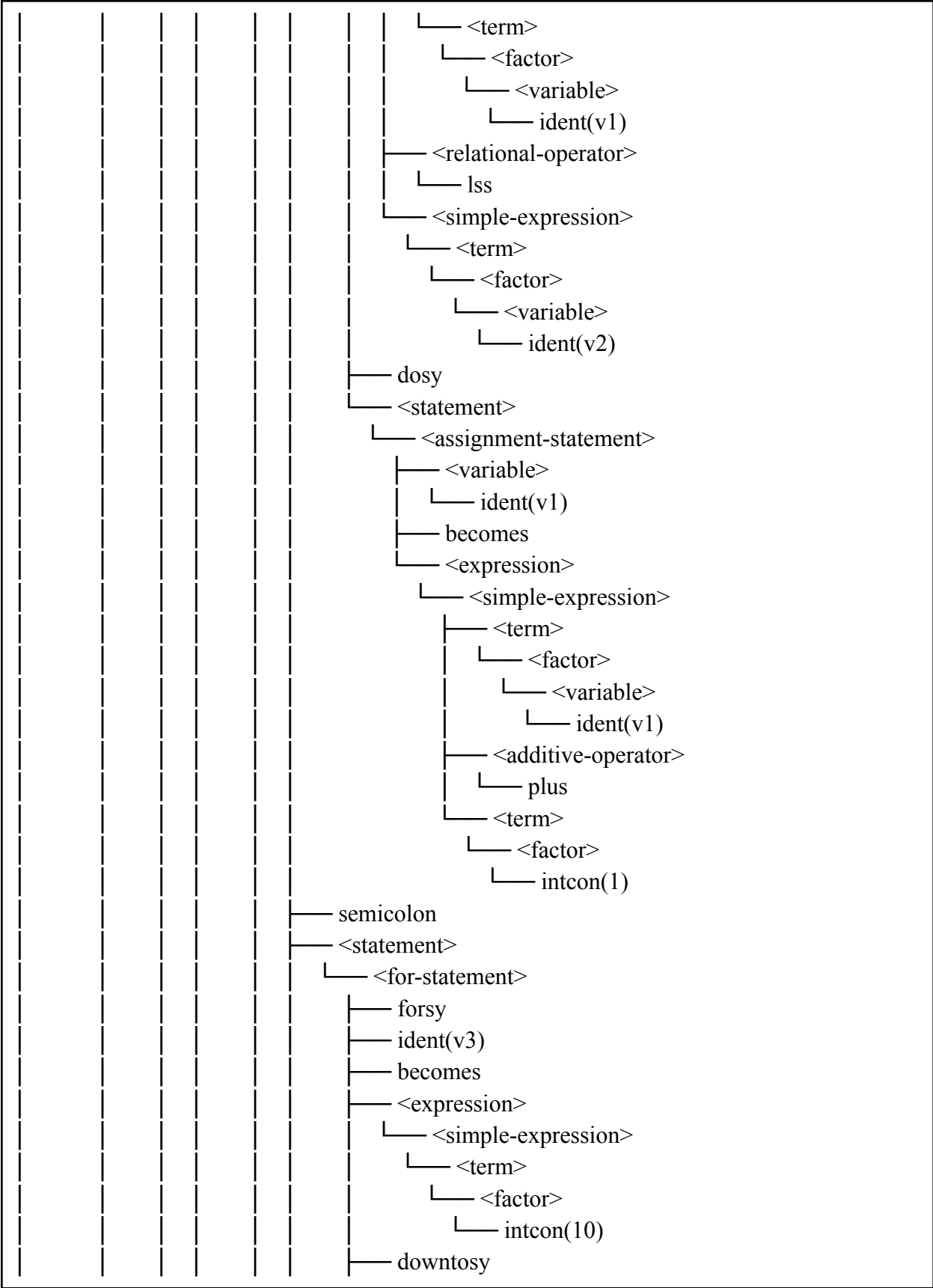
└─ <program-header>

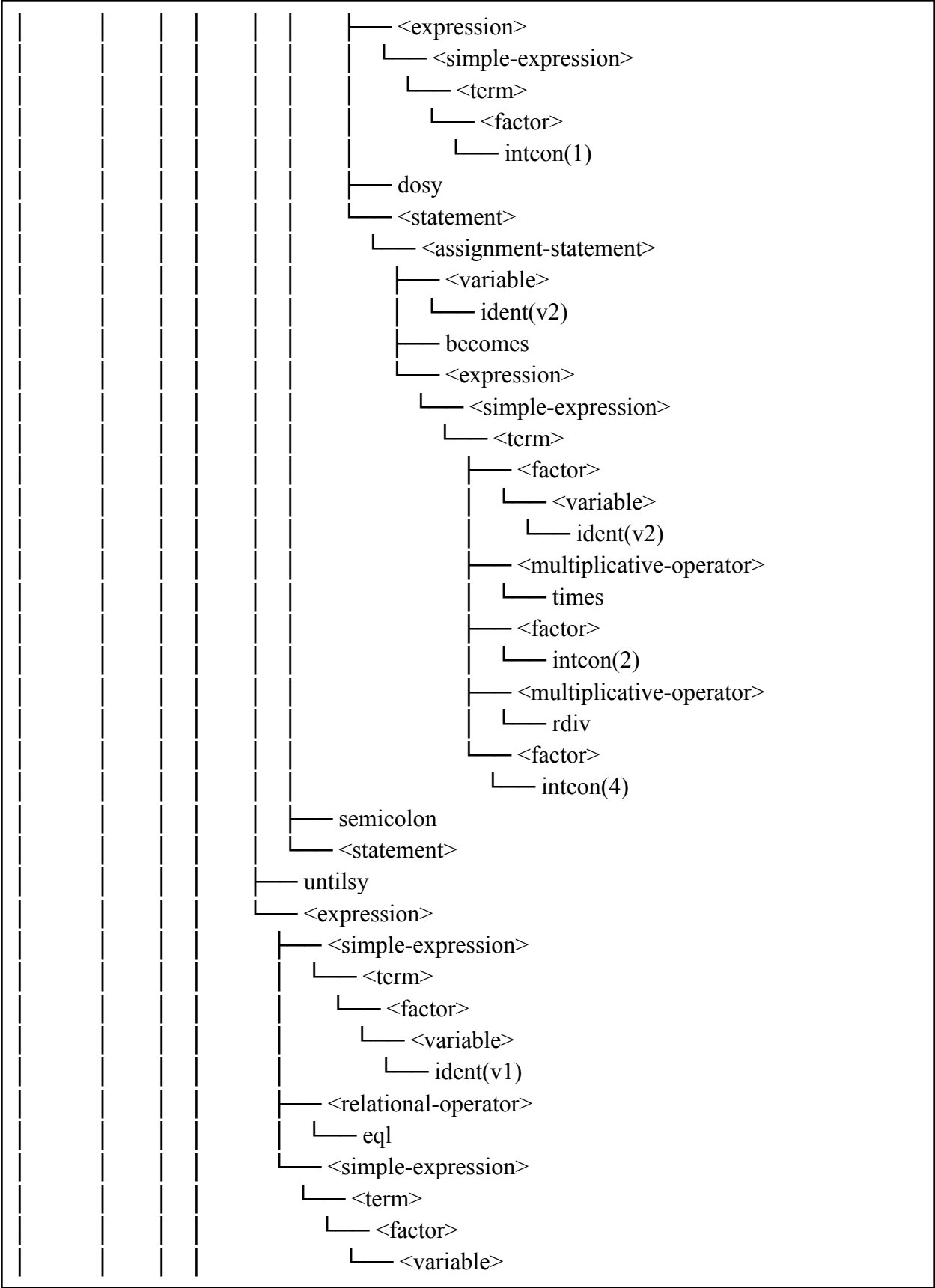


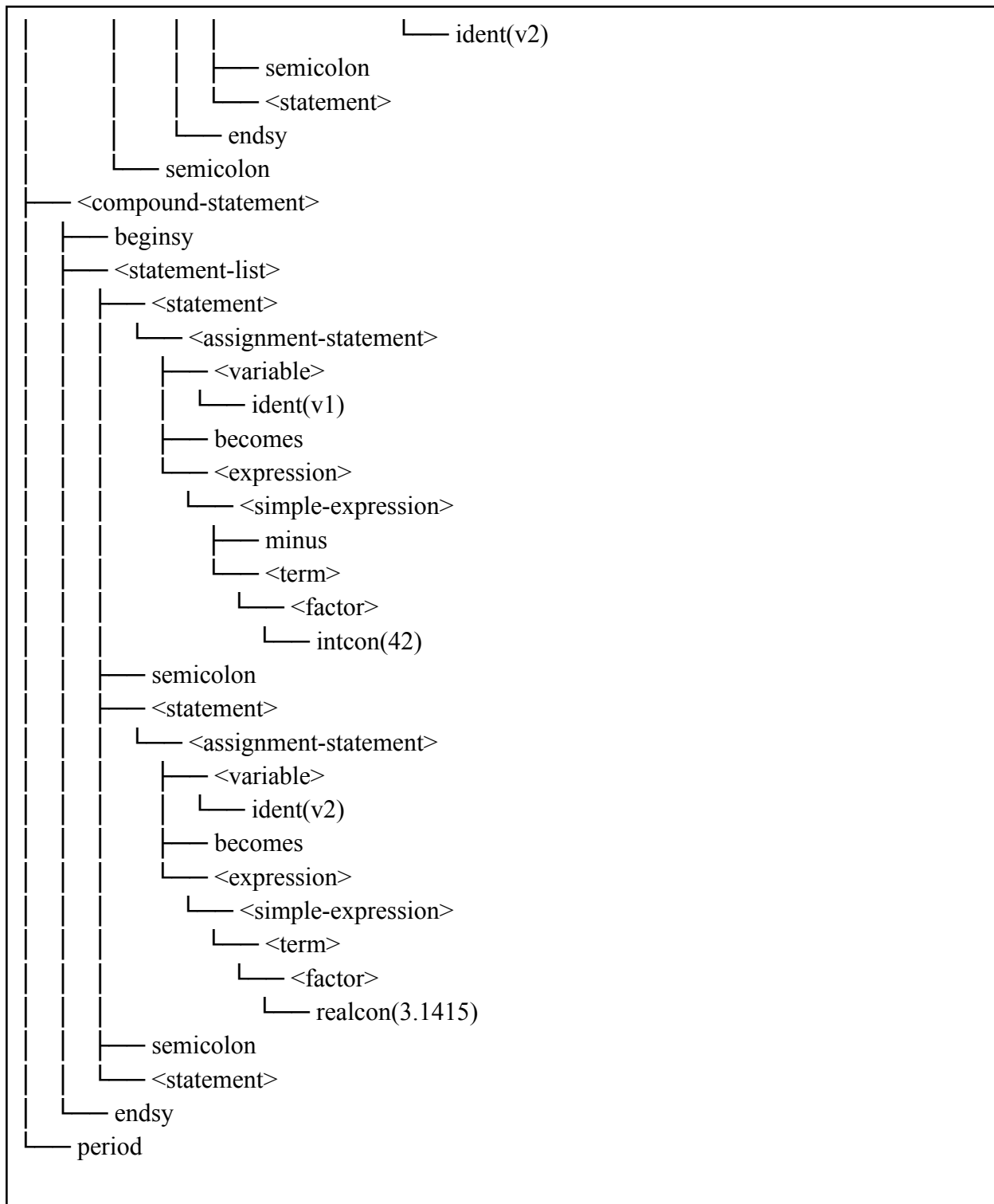












3.7 Test Case 7

Test Case berhasil, semua aturan produksi terpenuhi dengan baik

INPUT

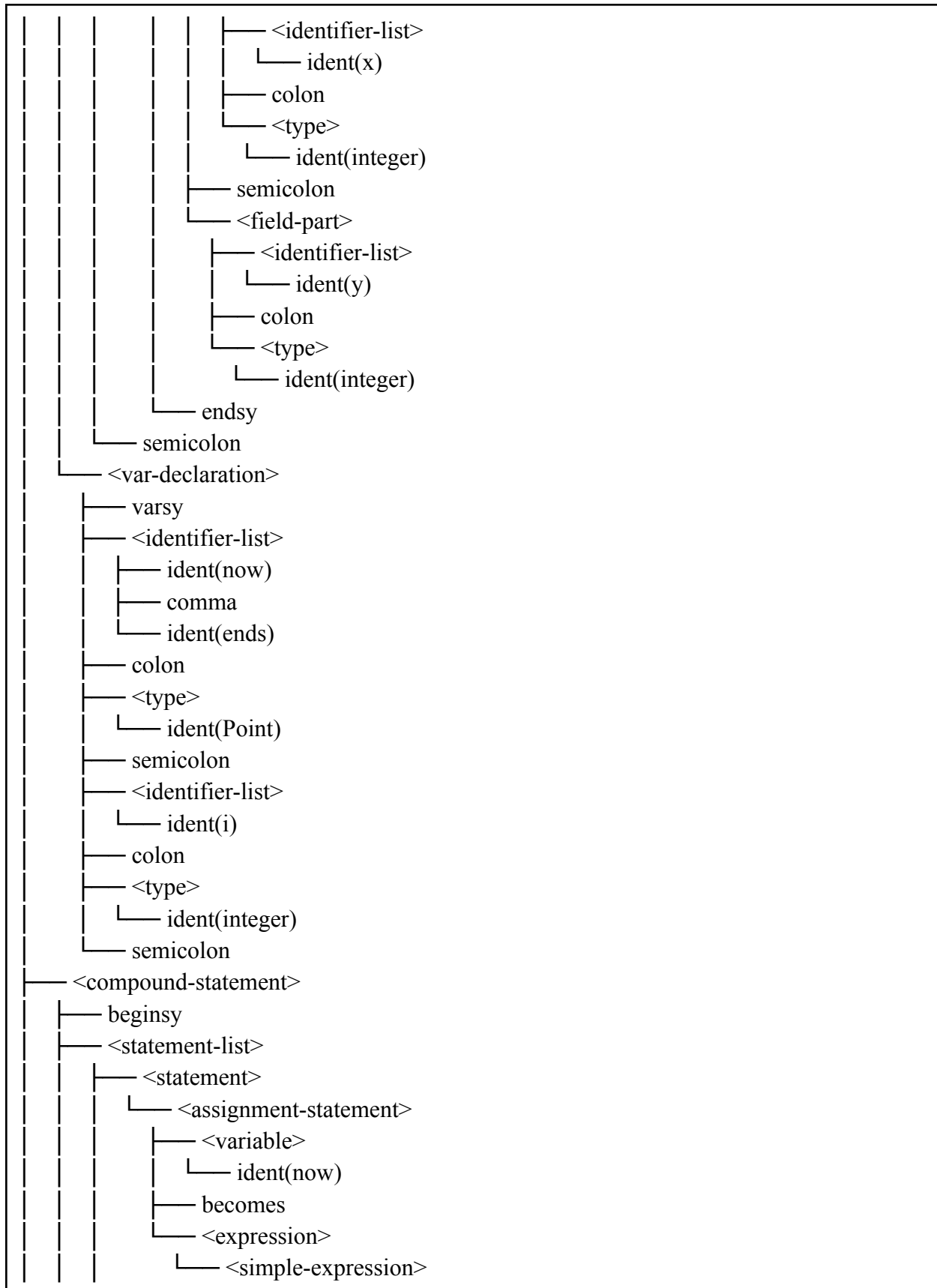
```
1  program Test4;  
2  
3  type Point ==  
4  record  
5  |   x: integer;  
6  |   y: integer  
7  end;  
8  
9  var  
10 |   now, ends: Point;  
11 |   i: integer;  
12  
13 begin  
14 |   now := Point(0, 0);  
15 |   ends := Point(10, 10);  
16 |   for i := 10 downto 0 do  
17 |       case i of  
18 |           x : now.x := now.x + 1;  
19 |           y : now.y := now.y + 1;  
20 |       end;  
21 |   print('Point = ', x, 'And', y);  
22 end.
```

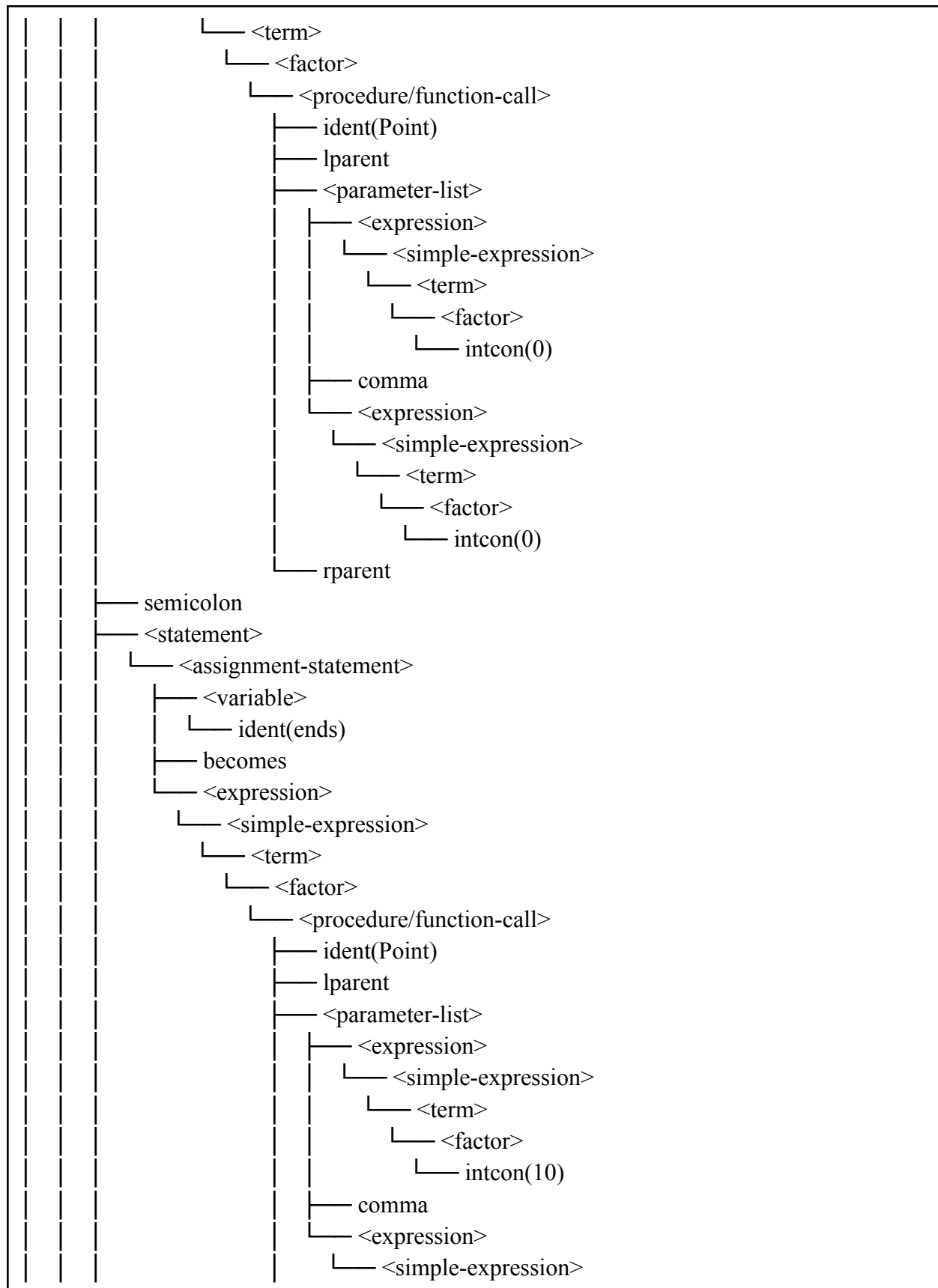
OUTPUT

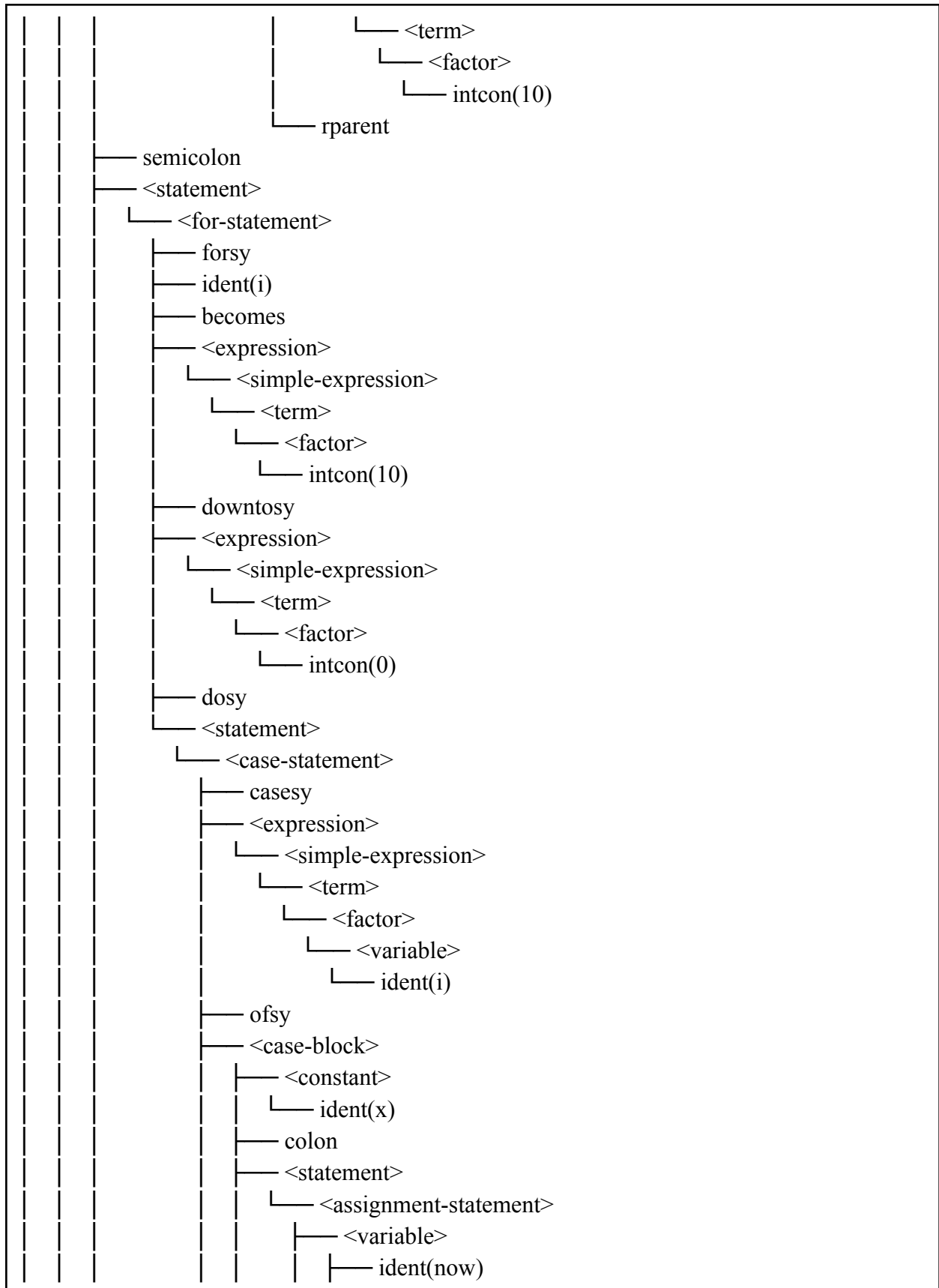
=== TIDAK ADA SYNTAX ERROR (PROGRAM VALID) ===

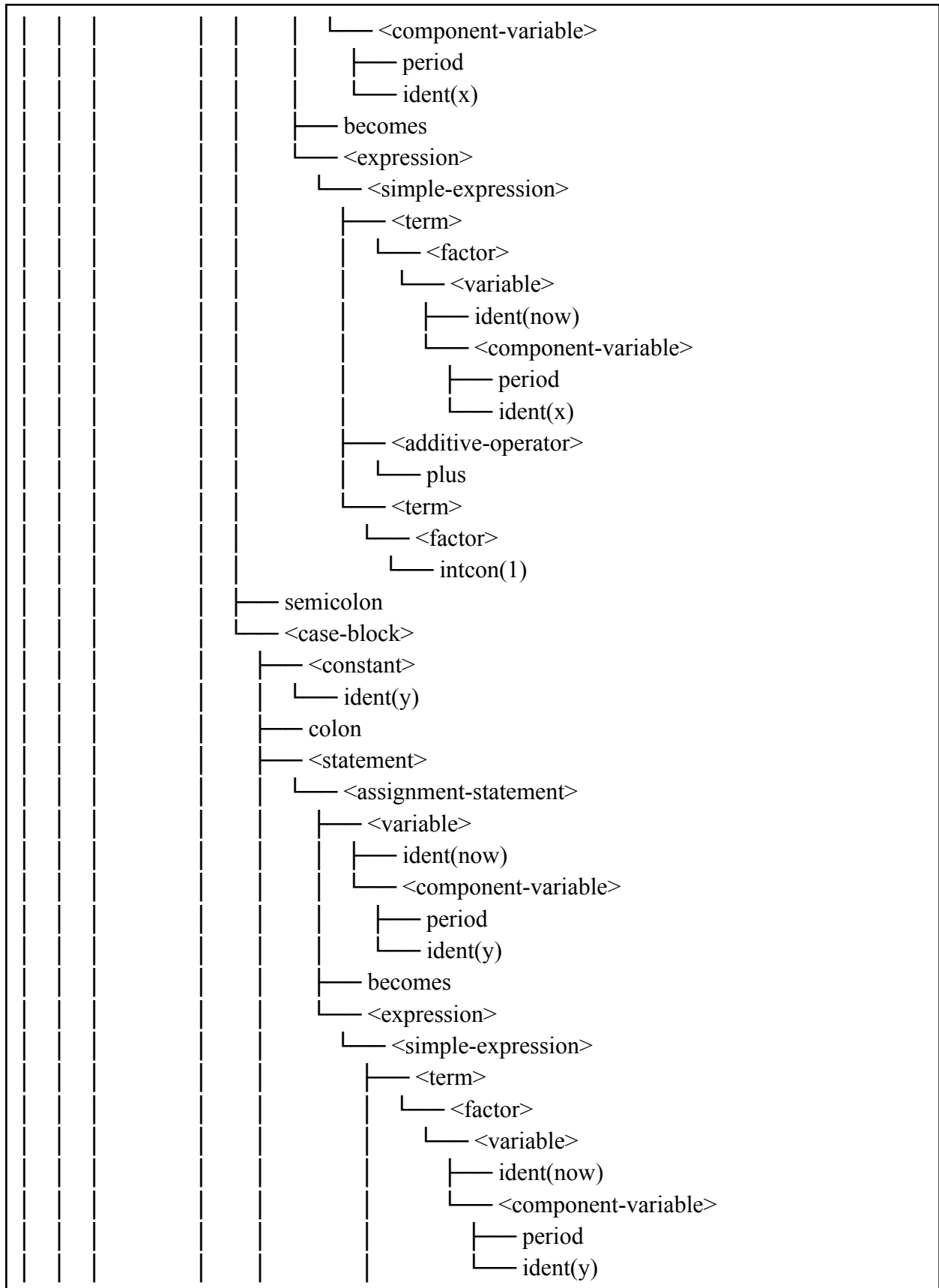
=== HASIL PARSE TREE ===

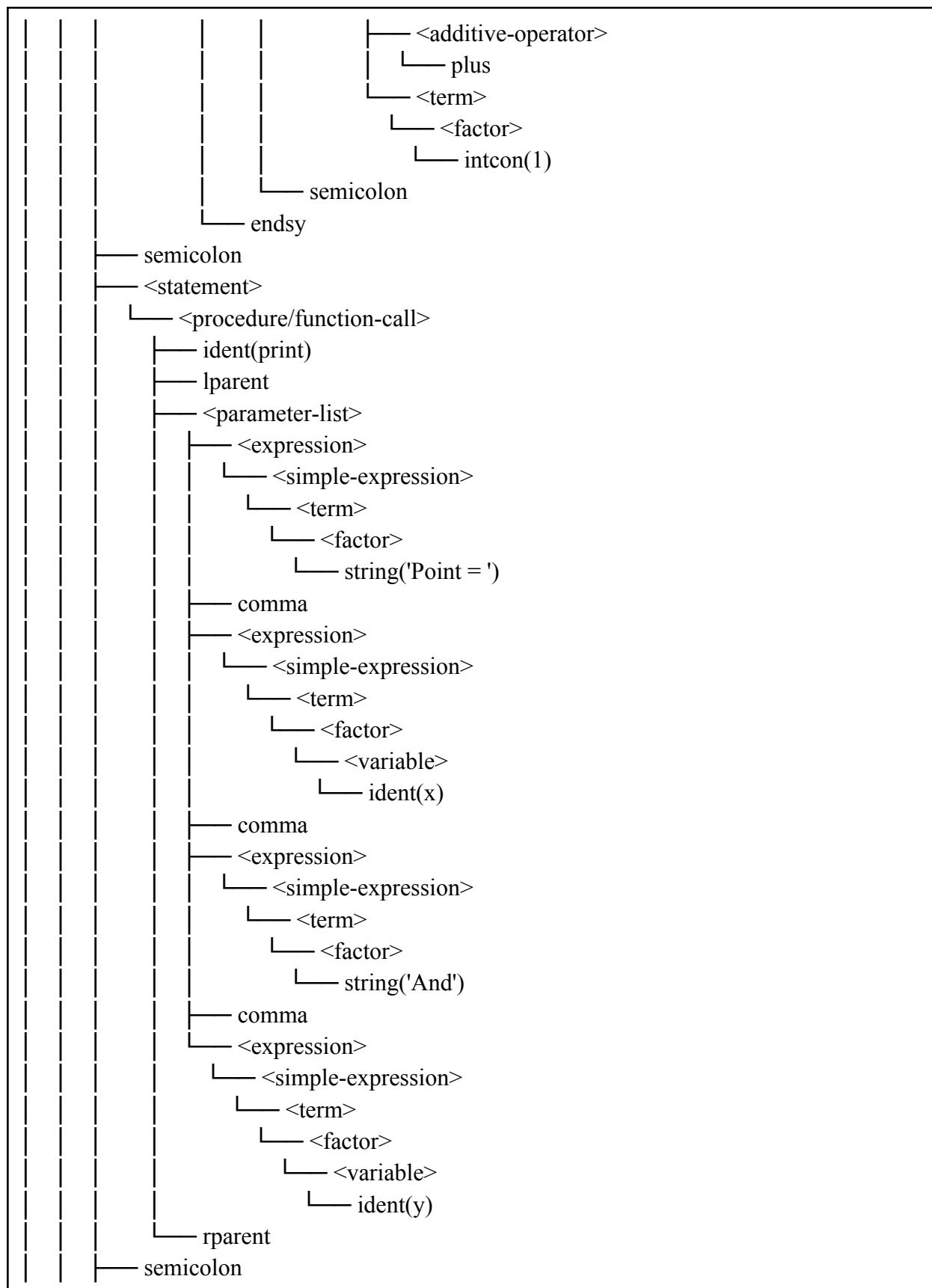
```
<program>  
├── <program-header>  
│   ├── programsy  
│   ├── ident(Test4)  
│   └── semicolon  
├── <declaration-part>  
│   ├── <type-declaration>  
│   │   ├── typesy  
│   │   ├── ident(Point)  
│   │   ├── eql  
│   │   ├── <type>  
│   │   └── <record-type>  
│   │       ├── recordsy  
│   │       └── <field-list>  
│   │           └── <field-part>  
└──
```













BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa *Syntax Analyzer* (Parser) pada sistem ini berhasil dibangun menggunakan pendekatan recursive descent untuk menyusun parse tree dari token hasil analisis leksikal. Parser mampu mengenali struktur program Arion subset, dimulai dari program header, bagian deklarasi, tipe data, subprogram, hingga compound statement sebagai badan program.

1. Parser telah mencakup berbagai konstruksi bahasa Arion, seperti deklarasi konstanta, tipe, variabel, prosedur, fungsi, array, range, enumerated type, dan record.
2. Parser juga mendukung berbagai statement dasar, seperti assignment, if, case, while, repeat, for, serta pemanggilan prosedur atau fungsi.
3. Selain itu, parser mampu melakukan parsing ekspresi yang melibatkan operator relasional, aditif, dan multiplikatif.
4. Dengan demikian, struktur sintaks program dapat direpresentasikan secara hierarkis melalui parse tree.

Secara keseluruhan, hasil implementasi menunjukkan bahwa parser telah mampu melakukan analisis sintaksis sesuai grammar yang dirancang serta menghasilkan representasi parse tree yang mendukung proses pengembangan lebih lanjut.

4.2. Saran

Untuk pengembangan selanjutnya, parser dapat dilengkapi dengan mekanisme *error recovery* yang lebih baik agar kesalahan sintaks tidak langsung menghentikan proses parsing, melainkan dapat dilacak dan disinkronkan ke token berikutnya. Selain itu, perlu dilakukan penyempurnaan terhadap cakupan grammar agar lebih lengkap dan konsisten dengan seluruh bentuk sintaks Arion yang ditargetkan.

BAB V

LAMPIRAN

Link Repository Github: <https://github.com/staplesmaster/SSS-Tubes-IF2224-2026>

Pembagian Tugas:

Nama	NIM	Bagian	Persentase
Geraldo Artemius	13524005	• Parse Type dan Statement • Laporan	20%
Mikhael Andrian Yonatan	13524051	• Error handling • ParseTree implementation	20%
Junior Natra Situmorang	13524055	• Parse Statement dan Expression • Laporan	20%
Reynard Nathanael	13524103	• Parse Statement • Laporan	20%
Nicholas Luis Chandra	13524105	• Parse Declaration • Laporan	20%

BAB VI

REFERENSI

- [1] GeeksforGeeks, “Types of Parsers in Compiler Design,” *GeeksforGeeks*, Jul. 26, 2019. Available: <https://www.geeksforgeeks.org/compiler-design/types-of-parsers-in-compiler-design/>
- [2] GeeksforGeeks, “Introduction to Syntax Analysis in Compiler Design,” *GeeksforGeeks*, Sep. 22, 2015. Available: <https://www.geeksforgeeks.org/compiler-design/introduction-to-syntax-analysis-in-compiler-design/>
- [3] GeeksforGeeks, “Parsing Introduction to Parsers,” *GeeksforGeeks*, Feb. 08, 2017. Available: <https://www.geeksforgeeks.org/compiler-design/introduction-of-parsing-ambiguity-and-parsers-set-1/>
- [4] GeeksforGeeks, “Recursive Descent Parser,” *GeeksforGeeks*, Apr. 02, 2019. Available: <https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>
- [5] GeeksforGeeks, “Parse Tree in Compiler Design,” *GeeksforGeeks*, Sep. 16, 2020. Available: <https://www.geeksforgeeks.org/compiler-design/parse-tree-in-compiler-design/>
- [6] “Parse Tree in Automata Theory,” *Tutorialspoint.com*, 2026. Available: https://www.tutorialspoint.com/automata_theory/automata_theory_parse_tree.htm
- [7] GeeksforGeeks, “Lexical Analysis and Syntax Analysis,” *GeeksforGeeks*, Jan. 12, 2023. Available: <https://www.geeksforgeeks.org/compiler-design/lexical-analysis-and-syntax-analysis/>
- [8] *Youtu.be*, 2026. Available: <https://youtu.be/OIKL6wFjFOo?si=aQ4151F4mNlWXJH9> . [Accessed: May 06, 2026]
- [9] *Youtu.be*, 2026. Available: <https://youtu.be/iddRD8tJi44?si=0nKOddqymLP9DMvU> . [Accessed: May 06, 2026]